

通过世界模型掌握多样领域

Danijar Hafner,^{1,2} Jurgis Pasukonis,¹ Jimmy Ba,² Timothy Lillicrap¹

摘要

开发能够跨应用领域学习解决任务的通用算法，是人工智能长期面临的核心挑战。现有强化学习算法通常可以直接用于与开发场景相近的任务；但一旦进入新的应用领域，往往需要大量人工经验和实验来重新配置。本文提出 DreamerV3，这是一种使用单一配置的通用算法，在 150 多个不同任务上超过了专门设计的方法。Dreamer 学习环境模型，并通过想象未来情景来改进行为。基于归一化、平衡和变换的稳健技术，使算法能够在不同领域中稳定学习。在默认配置下，Dreamer 是首个无需人类数据或课程、即可从零开始在 Minecraft 中收集钻石的算法。这一任务长期被视为人工智能中的难题，因为智能体必须从像素输入和开放世界中的稀疏奖励出发，探索具有前瞻性的策略。我们的结果表明，在不进行大规模调参实验的情况下，也可以解决具有挑战性的控制问题，从而提高强化学习的通用性。

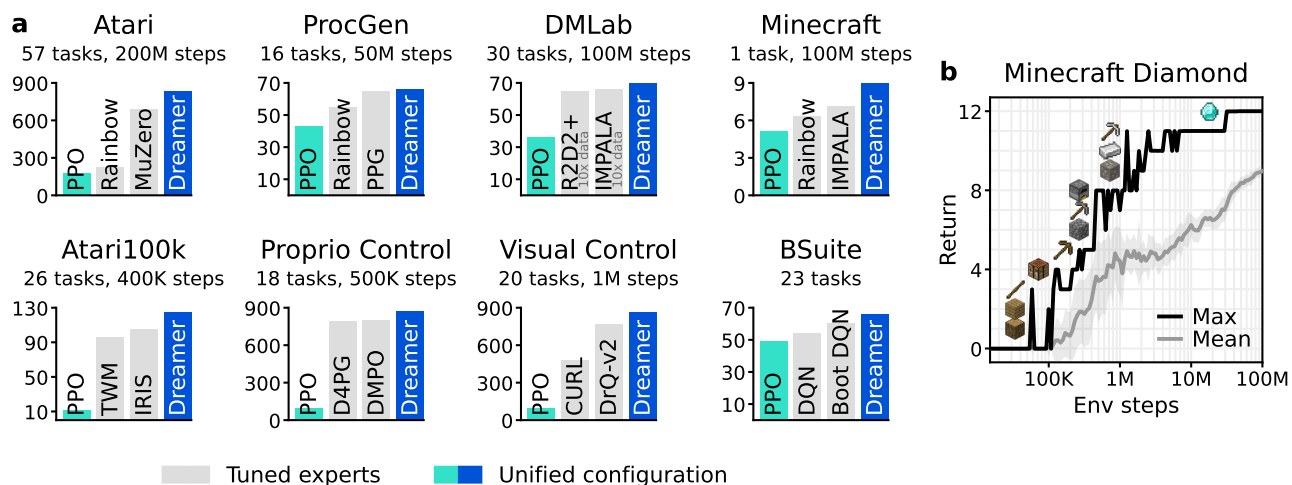


图 1: 基准摘要。a, 在所有领域使用固定超参数时，Dreamer 在多种基准和数据预算下超过调优后的专家算法，也超过一个高质量、通用性较强的 PPO 实现。b, 直接使用默认设置时，Dreamer 能够在流行电子游戏 Minecraft 中仅凭稀疏奖励从零开始学会获得钻石。这是人工智能中的长期难题，以往方法通常需要人类数据或领域专用启发式方法。

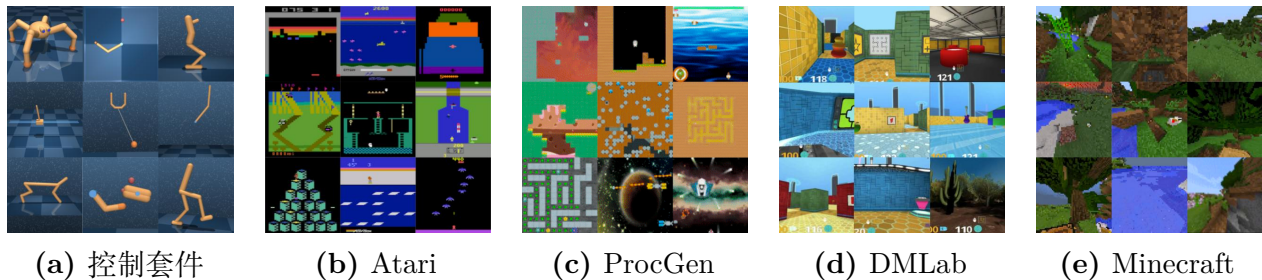


图 2: 实验中使用的多样化视觉领域。Dreamer 能在这些领域中取得成功, 覆盖机器人运动和操作任务、Atari 游戏、程序生成的 ProcGen 关卡、需要空间和时间推理的 DMLab 任务, 以及复杂且无限的 Minecraft 世界。我们也在非视觉领域上评估 Dreamer。

导言

强化学习使计算机能够通过与环境交互来解决任务, 例如在 Go 和 Dota 等游戏中达到甚至超过人类水平^{1,2}。它也是改进大型语言模型的重要工具, 可以帮助模型获得预训练数据之外的行为能力^{3,4}。PPO⁵ 已成为强化学习中的常用基线, 但在追求更高性能时, 研究者往往仍要针对具体任务设计更专门的算法。不同应用领域带来的困难并不相同, 包括连续控制⁶、离散动作^{7,8}、稀疏奖励⁹、图像输入¹⁰、空间环境¹¹ 以及棋盘游戏¹²。因此, 当强化学习算法被迁移到足够新的任务时, 例如从电子游戏迁移到机器人任务, 通常需要大量工程投入、领域经验和计算资源来调节超参数¹³。这种成本成为强化学习应用于新问题的瓶颈, 也限制了它在计算开销较高、无法反复调参的模型或任务上的使用。如果能够得到一种无需重新配置即可适应新领域的通用算法, 强化学习就更有可能进入广泛的实际应用场景。

本文提出 Dreamer, 这是一种在固定超参数下使用的通用算法, 能够在广泛领域中超过专门算法, 使强化学习更容易直接用于新问题。该算法的核心思想是学习一个世界模型, 使智能体能够基于感知经验预测未来^{14,15,16}。世界模型预测候选动作的结果, 价值网络评估这些结果的价值, 策略网络再选择能够取得较好结果的动作。尽管这一思路直观, 但如何稳定地学习并利用世界模型、进而获得强任务性能, 长期以来仍是开放问题¹⁷。Dreamer 通过一组基于归一化、平衡和变换的稳健技术来处理这一问题。实验表明, Dreamer 在图 2 所示的 150 多项任务上表现稳定; 随着模型规模和训练预算变化, 其性能提升也具有可预测性。更大的模型通常取得更高分数, 也往往需要更少的环境交互即可解决任务。

为进一步检验强化学习的能力边界, 我们关注近年来受到广泛研究的 Minecraft^{18,19,20}。围绕在 Minecraft 中自动学习收集钻石这一目标, 研究社区曾举办 MineRL 钻石竞赛^{*}。在没有人人类数据的情况下完成这一任务, 被普遍认为是人工智能中的难题: 开放世界带来稀疏奖励、困难探索、长时间跨度和程序化多样性等问题¹⁸。受这些困难限制, 先前方法通常依赖人类专家数据和特定领域课程^{19,20}。默认配置下的 Dreamer 是首个从零开始在 Minecraft 中收集钻石的算法。

学习算法

我们介绍第三代 Dreamer 算法^{21,22}。算法由三个神经网络组成: 世界模型预测候选动作的结果, critic 判断每个结果的价值, actor 选择能够达到高价值结果的动作。智能体与环境交互的同时, 这三个组件会从回放经验中并行训练。要跨领域成功工作, 三个组件都必须

^{*}MineRL 钻石竞赛于 2019、2020 和 2021 年举行, 并提供人类专家轨迹数据集: <https://minerl.io/diamond>。此后几年的竞赛转向了更广泛的任务。

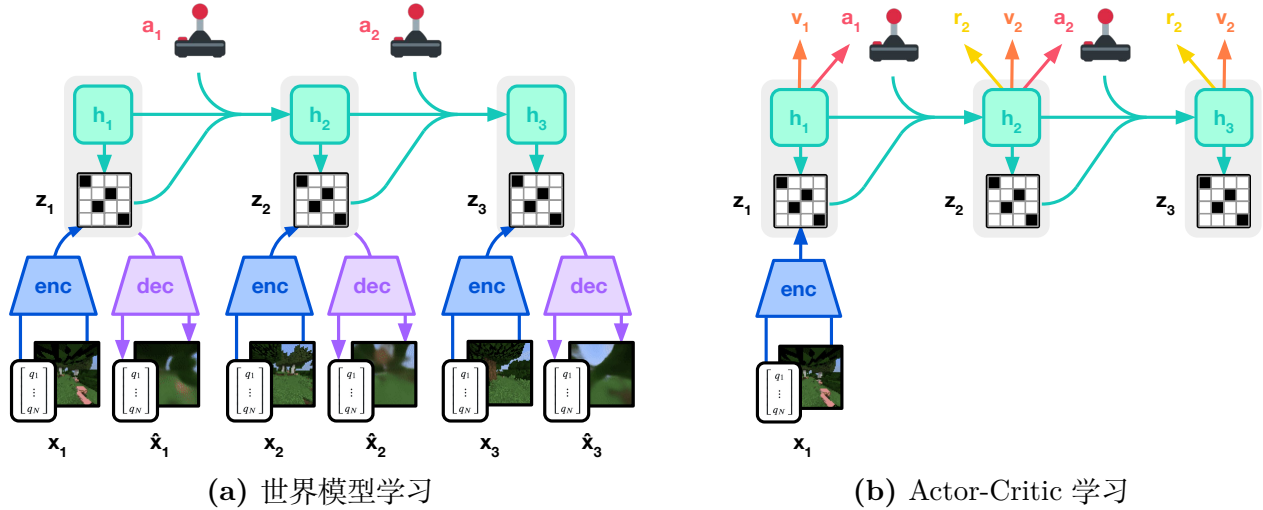


图 3: Dreamer 的训练流程。世界模型将感官输入编码为离散表示 z_t ；给定已执行动作 a_t 后，带循环状态 h_t 的序列模型预测这些表示。模型通过重建输入来塑造表征。Actor 和 critic 分别预测动作 a_t 与价值 v_t ，并从世界模型预测出的抽象表示轨迹中学习。

适应不同量级的信号，并在各自目标函数中稳健地平衡多个损失项。这很有挑战性：算法需要用固定超参数在多个差异很大的领域中学习，而不只是解决同一领域内的相似任务。本节介绍世界模型、critic 和 actor 及其稳健损失函数，并说明如何稳定预测未知量级的目标。

世界模型学习

世界模型通过自编码学习感知输入的紧凑表示²³，并通过预测候选动作下的未来表示和奖励来支持规划。我们将世界模型实现为循环状态空间模型 (RSSM)²⁴，如 图 3 所示。首先，编码器把感知输入 x_t 映射为随机表示 z_t 。随后，带有循环状态 h_t 的序列模型根据过去动作 a_{t-1} 预测这些表示的序列。 h_t 与 z_t 拼接后形成模型状态；模型从该状态预测奖励 r_t 、episode 是否继续的标志 $c_t \in \{0, 1\}$ ，并重建输入，以保证表示包含足够信息：

$$\text{RSSM} \begin{cases} \text{Sequence model:} & h_t = f_\phi(h_{t-1}, z_{t-1}, a_{t-1}) \\ \text{Encoder:} & z_t \sim q_\phi(z_t | h_t, x_t) \\ \text{Dynamics predictor:} & \hat{z}_t \sim p_\phi(\hat{z}_t | h_t) \\ \text{Reward predictor:} & \hat{r}_t \sim p_\phi(\hat{r}_t | h_t, z_t) \\ \text{Continue predictor:} & \hat{c}_t \sim p_\phi(\hat{c}_t | h_t, z_t) \\ \text{Decoder:} & \hat{x}_t \sim p_\phi(\hat{x}_t | h_t, z_t) \end{cases} \quad (1)$$

图 4 展示了世界模型的长期视频预测。对于图像输入，编码器和解码器使用卷积神经网络 (CNN)；对于向量输入，则使用多层感知机 (MLP)。动力学、奖励和 continue 预测器也采用 MLP。表示从一组 softmax 分布中采样，并在采样步骤上使用直通梯度^{25,22}。给定输入 $x_{1:T}$ 、动作 $a_{1:T}$ 、奖励 $r_{1:T}$ 和继续标志 $c_{1:T}$ 的序列批次，世界模型参数 ϕ 端到端优化，以最小化预测损失 $\mathcal{L}_{\text{pred}}$ 、动力学损失 $\mathcal{L}_{\text{dyn}}$ 和表示损失 $\mathcal{L}_{\text{rep}}$ ；对应权重为 $\beta_{\text{pred}} = 1$ 、 $\beta_{\text{dyn}} = 1$ 和 $\beta_{\text{rep}} = 0.1$ ：

$$\mathcal{L}(\phi) \doteq \mathbb{E}_{q_\phi} \left[\sum_{t=1}^T (\beta_{\text{pred}} \mathcal{L}_{\text{pred}}(\phi) + \beta_{\text{dyn}} \mathcal{L}_{\text{dyn}}(\phi) + \beta_{\text{rep}} \mathcal{L}_{\text{rep}}(\phi)) \right]. \quad (2)$$

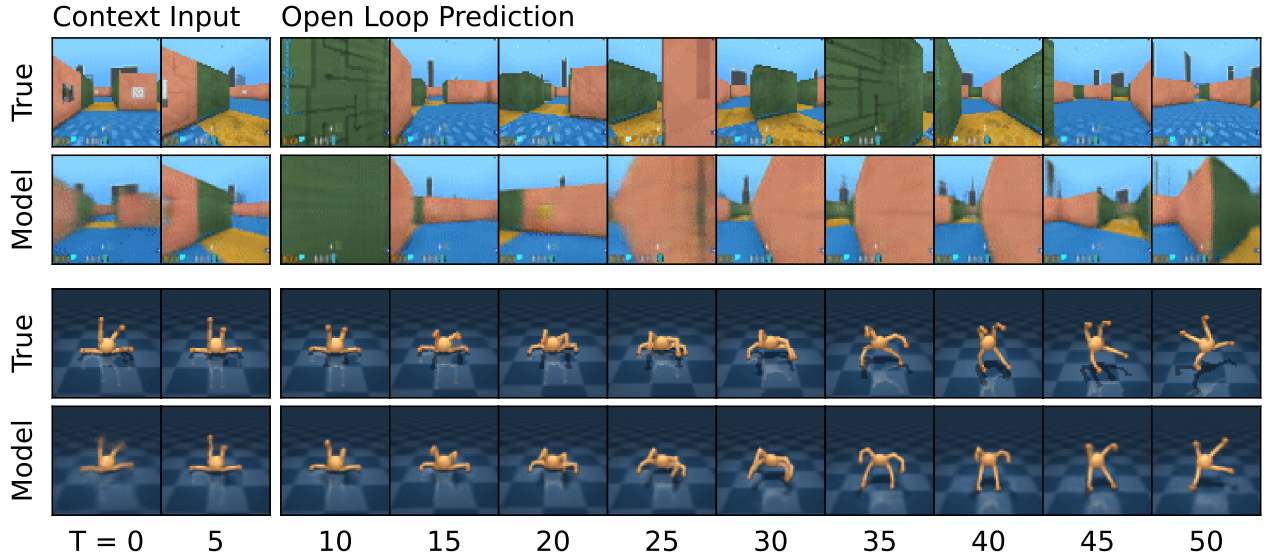


图 4: 对 DMLab 迷宫 (顶部) 和四角形机器人 (底部) 的多步视频预测。考虑到 5 个上下文图像和完整的动作序列, 模型预测未来会有 45 帧而无法获得中间图像. 世界模型了解每个环境的基本结构。

预测损失使用后文介绍的 symlog 平方损失训练解码器和奖励预测器, 并用逻辑回归训练 continue 预测器。动力学损失通过最小化预测器 $p_\phi(z_t | h_t)$ 与下一个随机表示 $q_\phi(z_t | h_t, x_t)$ 之间的 KL 散度, 训练序列模型预测下一个表示。表示损失则反过来促使表示变得更可预测, 从而允许我们在想象训练中使用因子化动力学预测器进行快速采样。这两个损失的区别在于停止梯度算子 $\text{sg}(\cdot)$ 的位置以及损失尺度。为避免出现退化解, 也就是动力学很容易预测但表示不包含足够输入信息, 我们采用 free bits²⁶, 将动力学损失和表示损失在 1 nat (约 1.44 bits) 以下截断。当这两个损失已经足够小时, 它们就被关闭, 使学习重点转向预测损失:

$$\begin{aligned}\mathcal{L}_{\text{pred}}(\phi) &\doteq -\ln p_\phi(x_t | z_t, h_t) - \ln p_\phi(r_t | z_t, h_t) - \ln p_\phi(c_t | z_t, h_t) \\ \mathcal{L}_{\text{dyn}}(\phi) &\doteq \max(1, \text{KL}[\text{sg}(q_\phi(z_t | h_t, x_t)) \parallel p_\phi(z_t | h_t)]) \\ \mathcal{L}_{\text{rep}}(\phi) &\doteq \max(1, \text{KL}[q_\phi(z_t | h_t, x_t) \parallel \text{sg}(p_\phi(z_t | h_t))])\end{aligned}\quad (3)$$

以往世界模型需要根据环境视觉复杂度, 对表示损失采用不同缩放²¹。复杂 3D 环境包含许多与控制无关的细节, 因此需要更强的正则化来简化表示并提高可预测性。而在具有静态背景、单个像素也可能影响任务的游戏, 则需要较弱正则化来保留细节。我们发现, 把 free bits 与较小的表示损失结合起来, 可以解决这一矛盾, 并允许跨领域使用固定超参数。此外, 我们使用后文介绍的 symlog 函数变换向量观测, 避免输入值和重建梯度过大, 从而进一步稳定表示损失的权衡。

早期实验中, 我们偶尔观察到 KL 损失尖峰, 这与深度变分自编码器中的报告一致²⁷。为避免这种情况, 我们把编码器和动力学预测器的 categorical 分布参数化为 1% 均匀分布与 99% 神经网络输出的混合, 使它们不可能变成确定性分布, 从而保证 KL 损失行为稳定。补充材料中包括更多的模型细节和超参数。

Critic 学习

Actor 和 critic 神经网络完全从世界模型预测出的抽象表示轨迹中学习行为¹⁴。与环境交互时, 我们直接从 actor 网络采样动作, 而不做前瞻规划。Actor 和 critic 都在模型状态

$s_t \doteq \{h_t, z_t\}$ 上运行，因此可以利用循环世界模型学到的马尔科夫表示。对于每个模型状态，actor 的目标是最大化回报 $R_t \doteq \sum_{\tau=0}^{\infty} \gamma^\tau r_{t+\tau}$ ，折扣因子为 $\gamma = 0.997$ 。为考虑超出预测时域 $T = 16$ 的奖励，critic 学习在当前 actor 行为下每个状态的回报分布近似²⁸：

$$\text{Actor: } a_t \sim \pi_\theta(a_t | s_t) \quad \text{Critic: } v_\psi(R_t | s_t) \quad (4)$$

从回放输入的表示开始，世界模型和 actor 生成一条想象轨迹，包括模型状态 $s_{1:T}$ 、动作 $a_{1:T}$ 、奖励 $r_{1:T}$ 和继续标志 $c_{1:T}$ 。由于 critic 预测的是一个分布，我们将其预测值 $v_t \doteq \mathbb{E}[v_\psi(\cdot | s_t)]$ 读作该分布的期望。为估计包含预测时域之外奖励的回报，我们计算自举 λ -return²⁹，将预测奖励和价值结合起来。critic 使用最大似然损失学习预测回报估计 R_t^λ 的分布：

$$\mathcal{L}(\psi) \doteq - \sum_{t=1}^T \ln p_\psi(R_t^\lambda | s_t) \quad R_t^\lambda \doteq r_t + \gamma c_t \left((1 - \lambda) v_t + \lambda R_{t+1}^\lambda \right) \quad R_T^\lambda \doteq v_T \quad (5)$$

一种简单做法是把 critic 参数化为正态分布，但回报分布可能是多模态的，而且不同环境下量级差异很大。为在这些条件下稳定并加速学习，我们将 critic 参数化为带指数间隔 bins 的 categorical 分布，使梯度尺度与预测目标量级解耦；具体做法见后文。为提升奖励难以预测环境中的价值预测，我们将 critic 损失同时用于想象轨迹和回放轨迹；想象轨迹的损失权重为 $\beta_{\text{val}} = 1$ ，从回放缓冲区采样的轨迹损失权重为 $\beta_{\text{repval}} = 0.3$ 。critic 的回放损失使用想象 rollout 起始状态处的想象回报 R_t^λ 作为 on-policy 价值标注，再结合回放奖励计算 λ -return。

由于 critic 回归的目标依赖自身预测，我们通过正则化 critic，使其接近自身参数指数移动平均所产生的输出，从而稳定学习。这类似于强化学习中常用的目标网络⁷，但仍允许我们使用当前 critic 网络计算回报。我们还注意到，训练开始时随机初始化的奖励预测器和 critic 可能产生很大的预测奖励，从而延迟学习启动。因此，我们将奖励预测器和 critic 的输出权重矩阵初始化为零，以缓解这一问题并加速早期学习。

Actor 学习

Actor 学习选择能够最大化回报的动作，同时通过熵正则项保持探索³⁰。然而，该正则项的合适尺度同时取决于环境中奖励的大小和频率。理想情况下，当奖励稀疏时，我们希望智能体探索更多；当奖励密集或容易获得时，则更偏向利用。与此同时，探索强度不应受到环境中任意奖励缩放的影响。因此，我们需要归一化回报尺度，同时保留奖励频率信息。

为在不同领域使用固定熵尺度 $\eta = 3 \times 10^{-4}$ ，我们将回报大致归一化到区间 $[0, 1]$ 。实践中，从回报中减去偏移量不会改变 actor 梯度，因此只需除以范围 S 。此外，为避免在稀疏奖励下放大函数近似带来的噪声，我们只缩小较大的回报幅度，而保持低于阈值 $L = 1$ 的小回报不变。我们对离散和连续动作都使用 Reinforce 估计器³¹，得到如下替代损失函数：

$$\mathcal{L}(\theta) \doteq - \sum_{t=1}^T \text{sg} \left((R_t^\lambda - v_\psi(s_t)) / \max(1, S) \right) \log \pi_\theta(a_t | s_t) + \eta \text{H} [\pi_\theta(a_t | s_t)] \quad (6)$$

回报分布可能是多模态的，并且包含离群值，特别是在随机化环境中，有些 episode 能达到的回报显著高于其他 episode。如果用观测到的最小和最大回报做归一化，回报尺度可能被压得过小，从而导致次优收敛。为对离群值保持稳健，我们使用回报批次中第 5th 到第 95th 百分位之间的范围，并用指数移动平均平滑该估计：

$$S \doteq \text{EMA}(\text{Per}(R_t^\lambda, 95) - \text{Per}(R_t^\lambda, 5), 0.99) \quad (7)$$

以往工作通常归一化优势函数⁵而不是回报，这会固定地强调回报最大化，而不管奖励是否容易达到。当奖励稀疏时，放大优势会同时放大噪声，噪声可能压过熵正则项并使探索

停滞。用标准差归一化奖励或回报也可能在稀疏奖励下失败，因为此时标准差接近零，会无视奖励大小而将其剧烈放大。约束优化方法会在状态平均意义上瞄准固定熵^{32,33}，不考虑可达回报；这种做法较稳健，但在稀疏奖励下探索较慢，在密集奖励下收敛值较低。这些方法都没有在跨领域设置中给出稳定的超参数。带分母下限的回报归一化克服了这些问题：它在稀疏奖励下探索更快，并能在多样领域中收敛到高性能。

稳健预测

重建输入以及预测奖励和回报具有挑战性，因为这些量的尺度会随领域而变化。用平方损失预测大目标可能导致发散，而绝对损失和 Huber 损失⁷又可能使学习停滞。另一方面，基于运行统计量归一化目标⁵会给优化引入非平稳性。我们提出 symlog 平方误差作为一个简单解决方案。具体来说，带输入 x 和参数 θ 的神经网络 $f(x, \theta)$ 学习预测目标 y 的变换版本。为读出网络预测 $\hat{y}$ ，我们应用逆变换：

$$\mathcal{L}(\theta) \doteq \frac{1}{2} (f(x, \theta) - \text{symlog}(y))^2 \quad \hat{y} \doteq \text{symexp}(f(x, \theta)) \quad (8)$$

直接使用对数变换无法处理可能为负的目标。因此，我们从双对称对数函数族中选择一个函数³⁴，称为 symlog；其逆函数称为 symexp：

$$\text{symlog}(x) \doteq \text{sign}(x) \ln(|x| + 1) \quad \text{symexp}(x) \doteq \text{sign}(x) (\exp(|x|) - 1) \quad (9)$$

symlog 函数会压缩大正值和大负值的幅度。与普通对数不同，它在保留输入符号的同时围绕原点对称。这使优化过程在需要时可以快速把网络预测移动到较大数值。同时，symlog 在原点附近近似恒等函数，因此不会影响已经足够小的目标的学习。

对于奖励或回报这类可能随机的目标，我们引入 symexp two-hot 损失。网络输出的是指数间隔 bins $b_i \in B$ 上 softmax 分布的 logits。预测值读作 bin 位置按预测概率加权后的平均值。重要的是，由于加权平均可以落在两个 bin 之间，网络仍然可以输出区间内任意连续值：

$$\hat{y} \doteq \text{softmax}(f(x))^T B \quad B \doteq \text{symexp}([-20 \quad \dots \quad +20]) \quad (10)$$

网络使用 two-hot 编码目标训练^{8,28}，这是 one-hot 编码在连续值上的推广。一个标量的 two-hot 编码是长度为 $|B|$ 的向量，除最接近该标量的两个 bin 索引 k 和 $k+1$ 外，其余项均为 0；这两个非零项之和为 1，离编码连续值更近的 bin 获得更高线性权重。随后，网络通过带软目标的 categorical cross-entropy 损失进行训练。注意，损失只依赖分配给 bin 的概率，而不依赖 bin 位置对应的连续值，因此梯度大小与目标大小解耦：

$$\mathcal{L}(\theta) \doteq - \text{twohot}(y)^T \log \text{softmax}(f(x, \theta)) \quad (11)$$

基于这些原则，Dreamer 对向量观测使用 symlog 变换，既用于编码器输入，也用于解码器目标；同时，对奖励预测器和 critic 使用 symexp two-hot 损失。我们发现，这些技术能够在多样领域中带来稳健且快速的学习。对于 critic 学习，已有工作提出过非对称变换方案³⁵，但我们发现它在跨领域平均效果上较弱。与替代方案相比，symlog 变换避免了截断大目标⁷、由归一化引入非平稳性⁵，以及在检测到新极值时调整网络权重³⁶。

结果

我们在固定超参数下，跨 8 个领域、超过 150 个任务评估 Dreamer 的通用性。实验旨在将 Dreamer 与文献中表现最强的方法比较；这些方法通常针对相应基准做了专门设计和调参。

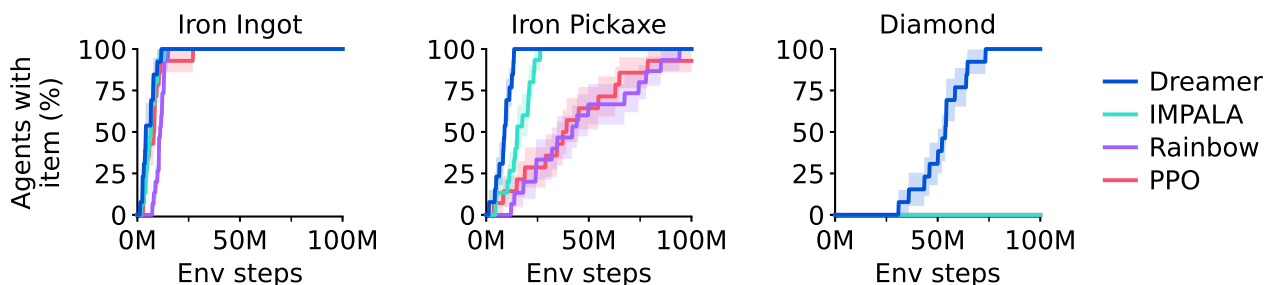


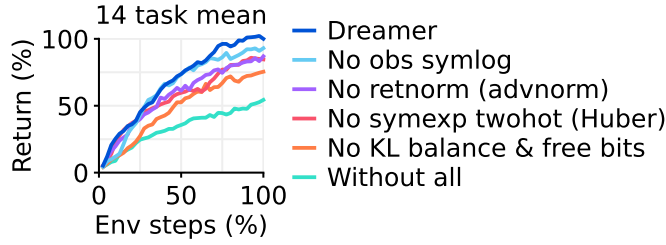
图 5: 在 Minecraft Diamond 任务中, 训练后的智能体发现最后三个物品的比例。虽然此前算法最多推进到铁镐, Dreamer 是唯一能够发现钻石的对比算法, 并且能稳定做到这一点。

我们还比较了一个高质量 PPO⁵ 实现。PPO 是标准强化学习算法, 并以稳健性著称。我们为 PPO 选择固定超参数, 使其尽可能在各领域取得较好表现, 同时复现 ProcGen 上已有公开 PPO 强结果³⁷。为推动强化学习边界, 我们还将 Dreamer 应用于具有挑战性的电子游戏 Minecraft, 并与此前强算法比较。最后, 我们分析 Dreamer 各组件的重要性, 以及它对不同模型规模和计算预算的稳健性。所有 Dreamer 智能体都分别在单张 Nvidia A100 GPU 上训练, 因此许多研究实验室都可以复现这些实验。能够复现全部结果的 Dreamer 公开实现可在项目网站获取。

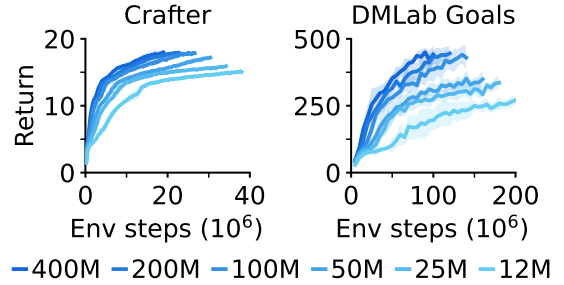
基准 我们在 8 个领域进行了广泛实证研究, 覆盖连续与离散动作、视觉与低维输入、稠密与稀疏奖励、不同奖励尺度、2D 与 3D 世界以及程序生成。图 1 汇总了基准结果, 表明 Dreamer 在多样化领域中超过了大量此前专家算法。其中, Dreamer 在所有领域均超过 PPO。

- **Atari** 这一成熟基准包含 57 个 Atari 2600 游戏, 预算为 200M 帧, 覆盖多种挑战³⁸。我们使用 sticky action 模拟器设置³⁹。Dreamer 在只使用少量计算资源的情况下超过了 MuZero 算法⁸。Dreamer 还超过了广泛使用的专家算法 Rainbow⁴⁰ 和 IQN⁴¹。
- **ProcGen** 该基准包含 16 个游戏, 具有随机关卡和视觉干扰, 用于测试智能体的稳健性和泛化能力⁴²。在 50M 帧预算内, Dreamer 匹配调优后的专家算法 PPG³⁷, 并超过 Rainbow^{42,40}。我们的固定超参数 PPO 智能体匹配官方高度调优 PPO 实现的公开分数³⁷。
- **DMLab** 该套件包含 30 个任务, 使用 3D 环境测试空间和时间推理能力⁴³。在 100M 帧内, Dreamer 超过可扩展的 IMPALA 和 R2D2+ 智能体在 1B 环境步处的表现³⁵, 相当于超过 1000% 的数据效率提升。这些基线并非为数据效率而设计, 但它们提供了此前大规模训练可达到性能的有价值参照。
- **Atari100k** 这一数据效率基准包含 26 个 Atari 游戏, 预算仅为 400K 帧, 相当于 2 小时游戏时间¹⁷。EfficientZero⁴⁴ 通过结合在线树搜索、优先回放和超参数调度保持最先进水平, 但它也会提前重置关卡以增加数据多样性, 使比较更复杂。在不引入这些复杂设置的情况下, Dreamer 超过了其余最佳方法, 包括基于 Transformer 的 IRIS 和 TWM 智能体、无模型方法 SPR, 以及 SimPL⁴⁵。
- **Proprio Control** 该基准包含 18 个控制任务, 动作为连续动作, 输入为本体感知向量, 预算为 500K 环境步⁴⁶。任务范围从经典控制和运动控制到机器人操作, 并包含稠密与稀疏奖励。Dreamer 在该基准上达到新的最优结果, 超过 D4PG、DMPO 和 MPO³³。
- **Visual Control** 该基准包含 20 个连续控制任务, 智能体只接收高维图像作为输入, 预算为 1M 环境步⁴⁶。Dreamer 在该基准上达到新的最优结果, 超过专门面向视觉环境并利用数据增强的 DrQ-v2 和 CURL⁴⁷。

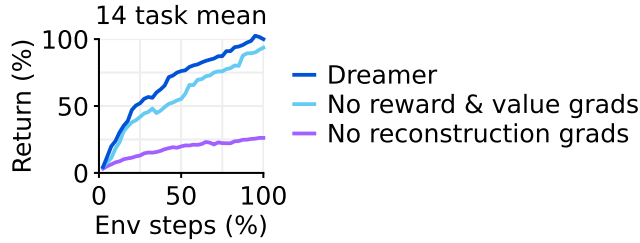
a Robustness techniques



c Model size scaling



b Learning signals



d Replay scaling

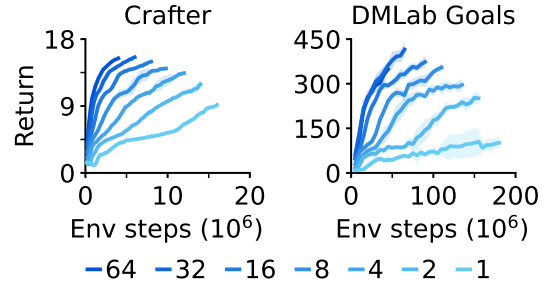


图 6: Dreamer 的消融实验与稳健扩展。**a**, 所有单项稳健性技术平均而言都提升 Dreamer 的性能, 尽管每项技术可能只影响部分任务; 各任务的训练曲线见补充材料。**b**, Dreamer 的性能主要依赖其世界模型的无监督重建损失, 这不同于多数主要依赖奖励和值预测梯度的先前算法^{7,5,8}。**c**, 随着模型规模从 12M 参数增至 400M 参数, Dreamer 的性能单调提升; 更大的模型通常提高任务性能, 也减少所需环境交互。**d**, 更高的回放比率可以可预测地提升 Dreamer 性能。结合模型规模, 这使实践者能够通过投入更多计算资源来提高任务性能和数据效率。

- **BSuite** 该基准包含 23 个环境、共 468 种配置, 专门用于测试信用分配、对奖励尺度和随机性的稳健性、记忆、泛化与探索⁴⁸。Dreamer 在该基准上达到新的最优结果, 超过 Boot DQN 等方法⁴⁹。相较此前算法, Dreamer 的提升在尺度稳健性类别中尤其明显。

Minecraft 在流行游戏 Minecraft 中收集钻石一直是人工智能中的长期难题^{18,19,20}。每个 episode 都发生在一个独特、随机生成且无限的 3D 世界中。episode 会持续到玩家死亡, 或最多持续 36,000 步 (即 30 分钟); 在此期间, 玩家需要通过采集资源和制作工具, 从稀疏奖励中发现通向钻石的 12 个物品序列。经验丰富的人类玩家通常需要约 20 分钟才能获得钻石²⁰。我们沿用先前工作的方块破坏设置¹⁹, 因为原始动作空间会让随机策略很难长时间保持按键。

由于该复杂领域训练时间较长, 在 Minecraft 上做大量调参并不现实。因此, 我们直接使用 Dreamer 的默认超参数。如图 1 and 5 所示, Dreamer 是首个无需 VPT 所需的人类数据²⁰、也无需自适应课程¹⁹, 即可从零开始在 Minecraft 中收集钻石的算法。我们在 Minecraft 上训练的所有 Dreamer 智能体都能在 100M 环境步内发现钻石。相比之下, 若干强基线虽然能推进到铁镐等高级物品, 没有任何一个发现钻石。

消融 在图 6 中, 我们在一组多样化的 14 个任务上消融稳健性技术和学习信号, 以理解它们的重要性。各个任务的训练曲线包含在补充材料中。结果表明, 所有稳健性技术都对性能有贡献, 其中最明显的是世界模型的 KL 目标, 其次是回报归一化, 以及用于奖励和值

预测的 symexp two-hot 回归。总体来看，每项技术都只在部分任务上起关键作用，并不会同等影响所有任务。

为研究世界模型的作用，我们消融 Dreamer 的学习信号：分别阻止任务相关的奖励和值预测梯度，或任务无关的重建梯度塑造其表征。不同于以往通常只依赖任务相关学习信号的强化学习算法^{7,8}，Dreamer 主要依赖其世界模型的无监督目标。这一发现表明，未来算法可以考虑利用无监督数据预训练。

尺度性质 为研究 Dreamer 能否稳健扩展，我们在 Crafter⁵⁰ 和一个 DMLab 任务⁴³ 上训练了 6 种模型规模（12M 到 400M 参数）以及不同回放比率。回放比率影响智能体执行梯度更新的次数。图 6 显示，在比较的模型规模和回放比率范围内，Dreamer 使用固定超参数仍能稳健学习。此外，增大模型规模会直接带来更高任务性能和更低数据需求。增加梯度步数也会进一步减少学习成功行为所需的环境交互。结果表明，Dreamer 在模型规模和回放比率变化下都能稳健学习，并且其性能会随计算资源增加而以较可预测的方式提升。

以往工作

开发通用算法长期以来一直是强化学习研究的目标。PPO⁵ 是使用最广泛的算法之一，相对稳健，但需要大量经验，而且性能往往低于专门方法。SAC³² 是连续控制中的常用选择，并利用经验回放提升数据效率；但实践中仍需要调参，尤其是熵系数，并且在高维输入下表现困难⁵¹。MuZero⁸ 使用价值预测模型进行规划，并已应用于棋盘游戏和 Atari；但作者没有发布实现，且算法包含多个复杂组件，因此复现有挑战。Gato⁵² 将一个大模型拟合到多任务专家示范上，但只有在存在专家数据时才适用。相比之下，Dreamer 能在固定超参数下掌握多样化环境，不需要专家数据，并且实现已开源。

Minecraft 也是近期研究重点。Microsoft 通过 MALMO⁵³ 发布了该成功游戏的免费研究版本。MineRL¹⁸ 提供多个竞赛环境，本文实验也以这些环境为基础。MineRL 竞赛通过多样化人类数据集，支持智能体探索并学习有意义的技能¹⁸。Voyager⁵⁴ 通过调用语言模型 API，在技术树中获得了与 Dreamer 相近深度的物品；但它运行在 MineFlayer 脚本层之上，该层是专门为游戏工程化设计的，并暴露高层动作。VPT²⁰ 基于承包者收集的键盘鼠标专家数据，通过行为克隆训练 Minecraft 智能体，并使用强化学习微调；其获得钻石需要 720 块 GPU 训练 9 天。相比之下，Dreamer 使用 MineRL 竞赛动作空间，在没有人类数据的情况下，仅用 1 块 GPU 训练 9 天，就能从稀疏奖励中自主学习收集钻石。

结论

我们介绍了第三代 Dreamer 算法。它是一种通用强化学习算法，可在固定超参数下跨多个差异很大的领域学习。Dreamer 在 150 多项任务上表现稳定，并能随着数据和计算预算增加而可预测地提升性能；这有助于减少将强化学习迁移到新问题时所需的调参成本。在默认配置下，Dreamer 是首个无需人类数据或课程学习、即可从零开始在 Minecraft 中采集钻石的算法。这一结果表明，学习得到的世界模型可以支撑复杂、长时程、稀疏奖励任务中的探索和控制。后续工作可以进一步研究如何从互联网视频中学习世界知识，如何用这类知识指导智能体训练，以及如何构建跨领域共享的单一世界模型，使智能体逐步积累更通用的知识和能力。

鸣谢 我们感谢穆罕默德·诺鲁齐、杰西·林、阿巴斯·阿卜杜勒马勒基、约翰·舒尔曼、亚当·科西奥雷克和奥莱赫·雷布金进行了富有洞察力的讨论。我们感谢博巴克·沙赫里亚里、丹尼斯·亚拉特斯、卡尔·科布比、与休伯特·索耶分享基线算法的训练曲线。我们感谢丹尼尔·富雷尔、安德鲁·陈和达克谢什·加兰巴为谷歌云基础设施提供帮助。

References

1. David Silver, Aja Huang, Chris J Maddison, Arthur Guez, Laurent Sifre, George Van Den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. Mastering the game of go with deep neural networks and tree search. *nature*, 529(7587):484, 2016.
2. OpenAI. OpenAI Five. <https://blog.openai.com/openai-five/>, 2018.
3. Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.
4. Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven Chu Hong Hoi. Coderl: Mastering code generation through pretrained models and deep reinforcement learning. *Advances in Neural Information Processing Systems*, 35:21314–21328, 2022.
5. John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
6. Timothy P Lillicrap, Jonathan J Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971*, 2015.
7. Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529, 2015.
8. Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, et al. Mastering atari, go, chess and shogi by planning with a learned model. *arXiv preprint arXiv:1911.08265*, 2019.
9. Max Jaderberg, Volodymyr Mnih, Wojciech Marian Czarnecki, Tom Schaul, Joel Z Leibo, David Silver, and Koray Kavukcuoglu. Reinforcement learning with unsupervised auxiliary tasks. *arXiv preprint arXiv:1611.05397*, 2016.
10. Ankesh Anand, Evan Racah, Sherjil Ozair, Yoshua Bengio, Marc-Alexandre Côté, and R Devon Hjelm. Unsupervised state representation learning in atari. *Advances in neural information processing systems*, 32, 2019.
11. Danny Driess, Ingmar Schubert, Pete Florence, Yunzhu Li, and Marc Toussaint. Reinforcement learning with neural radiance fields. *arXiv preprint arXiv:2206.01634*, 2022.
12. David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, et al. Mastering the game of go without human knowledge. *Nature*, 550(7676):354, 2017.
13. Marcin Andrychowicz, Anton Raichuk, Piotr Stańczyk, Manu Orsini, Sertan Girgin, Raphael Marinier, Léonard Hussenot, Matthieu Geist, Olivier Pietquin, Marcin Michalski, et al. What matters in on-policy reinforcement learning? a large-scale empirical study. *arXiv preprint arXiv:2006.05990*, 2020.

14. Richard S Sutton. Dyna, an integrated architecture for learning, planning, and reacting. *ACM SIGART Bulletin*, 2(4):160–163, 1991.
15. Chelsea Finn and Sergey Levine. Deep visual foresight for planning robot motion. In *2017 IEEE International Conference on Robotics and Automation (ICRA)*, pages 2786–2793. IEEE, 2017.
16. David Ha and Jürgen Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2018.
17. Lukasz Kaiser, Mohammad Babaeizadeh, Piotr Milos, Blazej Osinski, Roy H Campbell, Konrad Czechowski, Dumitru Erhan, Chelsea Finn, Piotr Kozakowski, Sergey Levine, et al. Model-based reinforcement learning for atari. *arXiv preprint arXiv:1903.00374*, 2019.
18. William H Guss, Cayden Codel, Katja Hofmann, Brandon Houghton, Noboru Kuno, Stephanie Milani, Sharada Mohanty, Diego Perez Liebana, Ruslan Salakhutdinov, Nicholay Topin, et al. The minerl competition on sample efficient reinforcement learning using human priors. *arXiv e-prints*, pages arXiv–1904, 2019.
19. Ingmar Kanitscheider, Joost Huizinga, David Farhi, William Hebgen Guss, Brandon Houghton, Raul Sampedro, Peter Zhokhov, Bowen Baker, Adrien Ecoffet, Jie Tang, et al. Multi-task curriculum learning in a complex, visual, hard-exploration domain: Minecraft. *arXiv preprint arXiv:2106.14876*, 2021.
20. Bowen Baker, Ilge Akkaya, Peter Zhokhov, Joost Huizinga, Jie Tang, Adrien Ecoffet, Brandon Houghton, Raul Sampedro, and Jeff Clune. Video pretraining (vpt): Learning to act by watching unlabeled online videos. *arXiv preprint arXiv:2206.11795*, 2022.
21. Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. *arXiv preprint arXiv:1912.01603*, 2019.
22. Danijar Hafner, Timothy Lillicrap, Mohammad Norouzi, and Jimmy Ba. Mastering atari with discrete world models. *arXiv preprint arXiv:2010.02193*, 2020.
23. Diederik P Kingma and Max Welling. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*, 2013.
24. Danijar Hafner, Timothy Lillicrap, Ian Fischer, Ruben Villegas, David Ha, Honglak Lee, and James Davidson. Learning latent dynamics for planning from pixels. *arXiv preprint arXiv:1811.04551*, 2018.
25. Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013.
26. Durk P Kingma, Tim Salimans, Rafal Jozefowicz, Xi Chen, Ilya Sutskever, and Max Welling. Improved variational inference with inverse autoregressive flow. *Advances in neural information processing systems*, 29, 2016.
27. Rewon Child. Very deep vaes generalize autoregressive models and can outperform them on images. *arXiv preprint arXiv:2011.10650*, 2020.
28. Marc G Bellemare, Will Dabney, and Rémi Munos. A distributional perspective on reinforcement learning. In *International Conference on Machine Learning*, pages 449–458. PMLR, 2017.

29. Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*. MIT press, 2018.
30. Ronald J Williams and Jing Peng. Function optimization using connectionist reinforcement learning algorithms. *Connection Science*, 3(3):241–268, 1991.
31. Ronald J Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning*, 8(3-4):229–256, 1992.
32. Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. *arXiv preprint arXiv:1801.01290*, 2018.
33. Abbas Abdolmaleki, Jost Tobias Springenberg, Yuval Tassa, Remi Munos, Nicolas Heess, and Martin Riedmiller. Maximum a posteriori policy optimisation. *arXiv preprint arXiv:1806.06920*, 2018.
34. J Beau W Webber. A bi-symmetric log transformation for wide-range data. *Measurement Science and Technology*, 24(2):027001, 2012.
35. Steven Kapturowski, Georg Ostrovski, John Quan, Remi Munos, and Will Dabney. Recurrent experience replay in distributed reinforcement learning. In *International conference on learning representations*, 2018.
36. Matteo Hessel, Hubert Soyer, Lasse Espeholt, Wojciech Czarnecki, Simon Schmitt, and Hado van Hasselt. Multi-task deep reinforcement learning with popart. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pages 3796–3803, 2019.
37. Karl W Cobbe, Jacob Hilton, Oleg Klimov, and John Schulman. Phasic policy gradient. In *International Conference on Machine Learning*, pages 2020–2027. PMLR, 2021.
38. Marc G Bellemare, Yavar Naddaf, Joel Veness, and Michael Bowling. The arcade learning environment: An evaluation platform for general agents. *Journal of Artificial Intelligence Research*, 47:253–279, 2013.
39. Marlos C Machado, Marc G Bellemare, Erik Talvitie, Joel Veness, Matthew Hausknecht, and Michael Bowling. Revisiting the arcade learning environment: Evaluation protocols and open problems for general agents. *Journal of Artificial Intelligence Research*, 61: 523–562, 2018.
40. Matteo Hessel, Joseph Modayil, Hado Van Hasselt, Tom Schaul, Georg Ostrovski, Will Dabney, Dan Horgan, Bilal Piot, Mohammad Azar, and David Silver. Rainbow: Combining improvements in deep reinforcement learning. In *Thirty-Second AAAI Conference on Artificial Intelligence*, 2018.
41. Will Dabney, Georg Ostrovski, David Silver, and Rémi Munos. Implicit quantile networks for distributional reinforcement learning. In *International conference on machine learning*, pages 1096–1105. PMLR, 2018.
42. Karl Cobbe, Chris Hesse, Jacob Hilton, and John Schulman. Leveraging procedural generation to benchmark reinforcement learning. In *International conference on machine learning*, pages 2048–2056. PMLR, 2020.
43. Charles Beattie, Joel Z Leibo, Denis Teplyashin, Tom Ward, Marcus Wainwright, Heinrich Küttler, Andrew Lefrancq, Simon Green, Víctor Valdés, Amir Sadik, et al. Deepmind lab. *arXiv preprint arXiv:1612.03801*, 2016.

44. Weirui Ye, Shaohuai Liu, Thanard Kurutach, Pieter Abbeel, and Yang Gao. Mastering atari games with limited data. *Advances in Neural Information Processing Systems*, 34: 25476–25488, 2021.
45. Vincent Micheli, Eloi Alonso, and François Fleuret. Transformers are sample efficient world models. *arXiv preprint arXiv:2209.00588*, 2022.
46. Yuval Tassa, Yotam Doron, Alistair Muldal, Tom Erez, Yazhe Li, Diego de Las Casas, David Budden, Abbas Abdolmaleki, Josh Merel, Andrew Lefrancq, et al. Deepmind control suite. *arXiv preprint arXiv:1801.00690*, 2018.
47. Denis Yarats, Rob Fergus, Alessandro Lazaric, and Lerrel Pinto. Mastering visual continuous control: Improved data-augmented reinforcement learning. *arXiv preprint arXiv:2107.09645*, 2021.
48. Ian Osband, Yotam Doron, Matteo Hessel, John Aslanides, Eren Sezener, Andre Saraiva, Katrina McKinney, Tor Lattimore, Csaba Szepesvari, Satinder Singh, et al. Behaviour suite for reinforcement learning. *arXiv preprint arXiv:1908.03568*, 2019.
49. Olivia Dizon-Paradis, Stephen Wormald, Daniel Capecci, Avanti Bhandarkar, and Damon Woodard. Investigating the practicality of existing reinforcement learning algorithms: A performance comparison. *Authorea Preprints*, 2023.
50. Danijar Hafner. Benchmarking the spectrum of agent capabilities. *arXiv preprint arXiv:2109.06780*, 2021.
51. Denis Yarats, Amy Zhang, Ilya Kostrikov, Brandon Amos, Joelle Pineau, and Rob Fergus. Improving sample efficiency in model-free reinforcement learning from images. *arXiv preprint arXiv:1910.01741*, 2019.
52. Scott Reed, Konrad Zolna, Emilio Parisotto, Sergio Gomez Colmenarejo, Alexander Novikov, Gabriel Barth-Maron, Mai Gimenez, Yury Sulsky, Jackie Kay, Jost Tobias Springenberg, et al. A generalist agent. *arXiv preprint arXiv:2205.06175*, 2022.
53. Matthew Johnson, Katja Hofmann, Tim Hutton, and David Bignell. The malmo platform for artificial intelligence experimentation. In *IJCAI*, pages 4246–4247. Citeseer, 2016.
54. Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.
55. Shengyi Huang, Rousslan Fernand Julien Dossa, Antonin Raffin, Anssi Kanervisto, and Weixun Wang. The 37 implementation details of proximal policy optimization. *The ICLR Blog Track 2023*, 2022.
56. Matt Hoffman, Bobak Shahriari, John Aslanides, Gabriel Barth-Maron, Feryal Behbahani, Tamara Norman, Abbas Abdolmaleki, Albin Cassirer, Fan Yang, Kate Baumli, et al. Acme: A research framework for distributed reinforcement learning. *arXiv preprint arXiv:2006.00979*, 2020.
57. Simon Schmitt, Matteo Hessel, and Karen Simonyan. Off-policy actor-critic with shared experience replay. In *International Conference on Machine Learning*, pages 8545–8554. PMLR, 2020.
58. Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. Prioritized experience replay. *arXiv preprint arXiv:1511.05952*, 2015.

59. Andy Brock, Soham De, Samuel L Smith, and Karen Simonyan. High-performance large-scale image recognition without normalization. In *International Conference on Machine Learning*, pages 1059–1071. PMLR, 2021.
60. Liu Ziyin, Zhikang T Wang, and Masahito Ueda. Laprop: Separating momentum and adaptivity in adam. *arXiv preprint arXiv:2002.04839*, 2020.
61. Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
62. Audrunas Gruslys, Will Dabney, Mohammad Gheshlaghi Azar, Bilal Piot, Marc Bellemare, and Remi Munos. The reactor: A fast and sample-efficient actor-critic agent for reinforcement learning. *arXiv preprint arXiv:1704.04651*, 2017.
63. Kyunghyun Cho, Bart Van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation. *arXiv preprint arXiv:1406.1078*, 2014.
64. Matthijs Van Keirsbilck, Alexander Keller, and Xiaodong Yang. Rethinking full connectivity in recurrent neural networks. *arXiv preprint arXiv:1905.12340*, 2019.
65. Marlos C Machado, Marc G Bellemare, Erik Talvitie, Joel Veness, Matthew Hausknecht, and Michael Bowling. Revisiting the arcade learning environment: Evaluation protocols and open problems for general agents. *Journal of Artificial Intelligence Research*, 61: 523–562, 2018.
66. Lasse Espeholt, Hubert Soyer, Remi Munos, Karen Simonyan, Volodymyr Mnih, Tom Ward, Yotam Doron, Vlad Firoiu, Tim Harley, Iain Dunning, et al. Impala: Scalable distributed deep-rl with importance weighted actor-learner architectures. *arXiv preprint arXiv:1802.01561*, 2018.

方法

基线

我们采用近端策略优化 (Proximal Policy Optimization, PPO) 算法⁵ 作为基线；PPO 已成为该领域的标准选择，适合在所有基准上用固定超参数与 Dreamer 比较。公开 PPO 实现很多，且任务性能差异较大⁵⁵。为确保比较能代表 PPO 在跨领域固定超参数下可达到的较高性能，我们选择 Acme 框架中的高质量 PPO 实现⁵⁶，并按 Huang et al.⁵⁵, Andrychowicz et al.¹³ 的建议选取 表 1 中的超参数；此外，我们还调整 epoch batch size，使其足以适配复杂环境⁴²，并调节学习率和熵系数。折扣因子与 Dreamer 保持一致，因为该设置跨领域表现良好，也是文献中的常见选择^{35,8}。我们采用表现优于替代架构的 IMPALA 网络架构⁴²，并将 minibatch size 设为单张 A100 GPU 可容纳的最大值。我们在 ProcGen 基准上验证 PPO 实现和超参数；该基准已有 PPO 作者报告的高度调优 PPO 结果³⁷。结果显示，我们的实现与该性能参考相当，或略高于它。

参数	数值
观测归一化	是
奖励归一化	是
奖励裁剪 (标准差)	10
Epoch batch	64×256
Epoch 数	3
Minibatch size	8
Minibatch length	256
策略信任区域	0.2
价值信任区域	否
优势归一化	是
熵惩罚系数	0.01
折扣因子	0.997
GAE lambda	0.95
学习率	3×10^{-4}
梯度裁剪	0.5
Adam epsilon	10^{-5}

表 1: PPO 在所有基准中使用的超参数。

对于 Minecraft，我们额外调参并运行 IMPALA 和 Rainbow 算法，因为文献中尚未报告从零开始的端到端学习能够成功完成该任务¹⁸。这些算法均采用 Acme 实现⁵⁶，并使用与 PPO 相同的 IMPALA 网络；我们调节了学习率和熵正则项。对于其余基准，我们与结果章节中引用的、文献报告的调优专家算法进行比较。

实现

经验回放 Dreamer 使用带在线队列的均匀回放缓冲区实现⁵⁷。具体而言，每个 minibatch 先由互不重叠的在线轨迹构成，再用从回放缓冲区均匀采样的轨迹补齐。数据收集时，我们将潜在状态写入回放缓冲区，用于在重放轨迹上初始化世界模型；训练 rollout 产生的新潜在状态也会写回缓冲区。优先经验回放⁵⁸ 被部分对比专家算法使用，我们也发现它可以提升 Dreamer 的性能。但为简化实现，本文实验采用均匀回放。

训练量通过回放比率参数化。回放比率表示每从环境收集一个时间步，需要训练多少个时间步，不计动作重复。将回放比率除以 minibatch 中的时间步数，再除以动作重复数，就得到梯度步数与环境步数的比例。例如，在 Atari 上动作重复为 4、batch 形状为 16×64 时，回放比率 32 对应每 128 个环境步执行 1 个梯度步，或在 200M 环境步上执行 1.5M 个梯度步。

优化器 我们使用自适应梯度裁剪 (Adaptive Gradient Clipping, AGC)⁵⁹：若某个张量的梯度范数超过对应权重矩阵 L2 范数的 30%，则对其进行裁剪，默认 $\epsilon = 10^{-3}$ 。AGC 将裁剪阈值与损失尺度解耦，因此可以改变损失函数或损失尺度，而无需重新调节裁剪阈值。裁剪后的梯度由 LaProp 优化器⁶⁰应用，设置 $\epsilon = 10^{-20}$ ，默认参数 $\beta_1 = 0.9$ 和 $\beta_2 = 0.99$ 。LaProp 先用 RMSProp 归一化梯度，再用动量平滑；不同于 Adam，它不是在原始梯度上同时计算动量和归一化项⁶¹。这一简单改动允许使用更小的 epsilon，并避免我们在 Adam 中偶尔观察到的不稳定性。

分布 编码器、动力学预测器和 actor 的分布均采用 unimix：即 99% 的预测 softmax 输出与 1% 的均匀分布混合⁶²，以避免零概率和无穷大的对数概率。奖励网络和 critic 网络输出定义在指数间隔分桶 $b \in B$ 上的 softmax 分布，并以 two-hot 编码目标训练：

$$\text{twohot}(x)_i \doteq \begin{cases} |b_{k+1} - x| / |b_{k+1} - b_k| & \text{if } i = k \\ |b_k - x| / |b_{k+1} - b_k| & \text{if } i = k + 1 \\ 0 & \text{else} \end{cases} \quad k \doteq \sum_{j=1}^{|B|} \delta(b_j < x) \quad (12)$$

two-hot 分布的输出权重初始化为零，以确保智能体在初始化时不会产生虚假的奖励和值预测。对于跨越多个数量级的分桶，计算 softmax 分布的期望时求和顺序会影响数值稳定性；正负分桶应分别从小到大求和，再相加。具体实现可参考源代码。

网络 图像先通过 stride 为 2 的卷积编码到 6×6 或 4×4 分辨率，再展平；解码时使用 stride 为 2 的转置卷积，并在输出端使用 sigmoid 激活。向量输入先做 symlog 变换，再用 3 层 MLP 编码和解码。actor 与 critic 也使用 3 层 MLP，奖励预测器和 continue 预测器使用 1 层 MLP。序列模型采用 GRU⁶³，其循环权重为 8 个块的块对角结构⁶⁴，这样可以容纳大量记忆单元，而不会使参数量和 FLOPs 呈二次增长。GRU 每个时间步的输入包括采样潜变量 z_t 、动作 a_t 和循环状态的线性嵌入，以便在不同块之间交换信息。

基准

协议 表 2 汇总了各基准设置；对于已有公认协议的基准，我们遵循其标准评估流程。Atari³⁸ 包含 57 个任务，并使用 sticky actions 设置⁶⁵。用于归一化分数的随机和人类参考分数在文献中并不完全一致，我们采用最常见的参考值，并在 表 6 中列出。DMLab⁴³ 包含 30 个任务⁶⁶，我们使用固定动作空间^{36,35}。由于像部分先前工作那样运行 10B 步不可行，我们在 100M 步处评估。已有公开基线在 100M 步时表现较弱，因此我们改与其 1B 步结果比较，相当于给这些基线 10× 的数据优势。ProcGen 使用 hard 难度和 unlimited level set⁴²。先前工作在不同步数预算下比较^{42,37}；由于计算成本且没有动作重复，我们在 50M 步处比较。对于只使用稀疏奖励的 Minecraft Diamond，我们建立评估协议，在 100M 环境步处报告 episode return，这大约对应 100 天游戏内时间。Atari100k¹⁷ 包含 26 个任务，预算为 400K 环境步，即动作重复后 100K 步。先前工作使用过多种环境设置，表 10 对此做了汇总；本文采用最初提出该基准时的环境设置。Visual Control^{46,21} 包含 20 个任务，动作重复为 2。Proprioceptive Control 采用相同协议，但由于先前工作中可用基线有限，我们排除两个四足机器人任务⁴⁷。

基准	任务数	环境步数	动作重复	环境实例	回放比率	GPU 天数	模型规模
Minecraft	1	100M	1	64	32	8.9	200M
DMLab	30	100M	4	16	32	2.9	200M
ProcGen	16	50M	1	16	64	16.1	200M
Atari	57	200M	4	16	32	7.7	200M
Atari100K	26	400K	4	1	128	0.1	200M
BSuite	23	—	1	1	1024	0.5	200M
Proprio Control	18	500K	2	16	512	0.3	12M
Visual Control	20	1M	2	16	512	0.1	12M

表 2: 基准概述。所有智能体均在单张 Nvidia A100 GPU 上训练。

环境实例 在早期实验中，我们观察到 Dreamer 和 PPO 的性能都对环境实例数量较为稳健。根据训练机器上可用的 CPU 资源，我们默认使用 16 个环境实例。对于 BSuite，基准要求使用单个环境实例。Atari100K 也使用单个环境实例，因为该基准预算为 400K 环境步，而 Atari 的最大 episode 长度原则上为 432K 环境步。对于 Minecraft，由于环境步进较慢，我们使用 64 个环境并通过远程 CPU worker 加速实验。

随机种子和误差带 每个基准上，我们为 Dreamer 和 PPO 各运行 5 个随机种子；例外情况是 ProcGen 因计算限制只运行 1 个种子，BSuite 按基准要求运行 10 个种子，Minecraft 运行 10 个种子以可靠报告获得钻石的运行比例。所有曲线均显示跨种子的平均值，阴影表示一个标准差。

计算设置 本文中的所有 Dreamer 和 PPO 智能体都在单张 Nvidia A100 GPU 上训练。Dreamer 默认使用 200M 参数规模的模型。在两个控制套件上，Dreamer 使用速度明显更快的 12M 模型即可达到相同表现，因而更容易被研究者复现。回放比率控制计算成本与数据效率之间的权衡；相关分析见 图 6，并据此为每个基准选择与步数预算匹配的设置。

模型规模

为适配不同计算预算，并分析不同模型规模下的稳健性，我们定义了从 12M 到 400M 参数的一组模型，见 表 3。这些规模由模型维度参数化，模型维度约按 1.5 倍增长，在 2 的幂次与 2 的幂次乘以 1.5 之间交替，从而得到满足硬件效率要求的 8 的倍数张量形状。不同网络组件的规模都由模型维度派生。MLP 的隐藏单元数等于模型维度。序列模型的循环单元数为模型维度的 8 倍，并分成 8 个与 MLP 同规模的块。最靠近数据的卷积编码器和解码器层使用比模型维度少 $16\times$ 的通道数。每个潜变量也使用比模型维度少 $16\times$ 的代码数。隐藏层数量和潜变量数量在所有模型规模中保持不变。所有超参数，包括学习率和 batch size，都在不同模型规模之间保持固定。

参数	12M	25M	50M	100M	200M	400M
隐藏维度 (d)	256	384	512	768	1024	1536
循环单元数 ($8d$)	1024	3072	4096	6144	8192	12288
基础 CNN 通道数 ($d/16$)	16	24	32	48	64	96
每个潜变量的代码数 ($d/16$)	16	24	32	48	64	96

表 3: Dreamer 模型规模。MLP 隐藏单元数定义模型维度，循环单元数、卷积通道数和每个潜变量的代码数均由该维度派生。层数和潜变量数量在不同模型规模之间保持不变。

Dreamer 前代方法

本文提出 Dreamer 系列的第三代算法；在需要区分时，我们称其为 DreamerV3。DreamerV1²¹ 主要面向连续控制，DreamerV2²² 在 Atari 上超过人类表现，而 DreamerV3 支持在多样化基准上开箱即用地学习。

DreamerV3 引入的主要改动如下：

- 稳健性技术：观测 symlog、KL balance 和 free bits、对所有分类分布使用 1% unimix、百分位回报归一化，以及用于奖励头和 critic 的 symexp two-hot 损失。
- 网络架构：Block GRU、RMSNORM 归一化、SiLU 激活。
- 优化器：自适应梯度裁剪（AGC）以及 LaProp（先 RMSProp 后动量）。
- 回放缓冲区：更大的容量、在线队列、存储并更新潜在状态。

超参数

名称	符号	数值
通用设置		
回放容量	—	5×10^6
Batch size	B	16
Batch length	T	64
激活与归一化	—	RMSNorm + SiLU
学习率	—	4×10^{-5}
梯度裁剪	—	AGC(0.3)
优化器	—	LaProp($\epsilon = 10^{-20}$)
世界模型		
重建损失系数	β_{pred}	1
动力学损失系数	β_{dyn}	1
表征损失系数	β_{rep}	0.1
潜变量 unimix	—	1%
Free nats	—	1
Actor-Critic		
想象时域	H	15
折扣时域	$1/(1 - \gamma)$	333
Return lambda	λ	0.95
Critic 损失系数	β_{val}	1
Critic 回放损失系数	β_{repval}	0.3
Critic EMA 正则项	—	1
Critic EMA 衰减	—	0.98
Actor 损失系数	β_{pol}	1
Actor 熵正则项	η	3×10^{-4}
Actor unimix	—	1%
Actor RetNorm 尺度	S	$\text{Per}(R, 95) - \text{Per}(R, 5)$
Actor RetNorm 下限	L	1
Actor RetNorm 衰减	—	0.99

表 4: Dreamer 超参数。所有基准均使用相同设置，包括本体感知和视觉输入、连续和离散动作、2D 和 3D 领域。我们不使用超参数退火、优先回放、权重衰减或 dropout。

Minecraft

游戏描述 Minecraft 每月活跃用户约 1 亿，是全球最受欢迎的电子游戏之一。游戏包含程序生成的 3D 世界，地貌由平原、森林、丛林、山地、沙漠、针叶林、雪原、冰刺、沼泽、草原、恶地、海滩、石岸、河流和海洋等不同生物群系构成。世界由边长约 1 米的方块组成，玩家可以破坏和放置这些方块。游戏中约有 30 种生物，玩家可以与其交互或战斗。玩家可以通过收集资源，使用 350 多种配方合成新物品，并沿技术树推进，同时还要保证安全和食物供应以便生存。Minecraft 中可以设定许多任务；作为第一步，研究社区主要关注获取钻石这一代表性任务。钻石是位于地下深处的稀有物品，玩家必须沿技术树逐步推进才能获得。

学习环境 我们在 MineRL 之上构建 Minecraft Diamond 环境，用于定义平坦的分类动作空间，并修复在人类试玩中发现的原始环境问题。例如，破坏钻石矿石时，物品有时会直接进入物品栏，有时则需要从地面拾取。原始环境会在破坏钻石矿石时结束 episode，导致许多成功回合在拾取物品之前终止，因此无法得到奖励。我们移除了这一提前终止条件，改为在玩家死亡或达到 36,000 步时结束 episode；在 20Hz 控制频率下，这对应 30 分钟。另一个问题是跳跃动作必须持续超过一个控制步才能触发跳跃。我们通过在后台保持按键 200ms 来解决。该环境基于 MineRL v0.4.4¹⁸ 构建，支持抽象合成动作；Minecraft 版本为 1.11.2。

观测和奖励 智能体观测 $64 \times 64 \times 3$ 的第一人称图像、包含 400 多种物品数量的物品栏计数向量、episode 开始以来各物品最大持有数量向量（用于指示已达成的里程碑）、表示当前装备物品的 one-hot 向量，以及生命值、饥饿值和氧气值等标量输入。我们沿用 MineRL 竞赛环境的稀疏奖励结构¹⁸，对通向钻石的 12 个里程碑给予奖励，包括获得原木、木板、木棍、工作台、木镐、圆石、石镐、铁矿石、熔炉、铁锭、铁镐和钻石。每个物品的奖励在每个 episode 中只发放一次，因此智能体必须学会多次收集某些物品，才能达成下一个里程碑。为便于解释回报，我们对每个里程碑给予 +1 奖励，而不是按物品价值缩放奖励。此外，每失去一颗心给予 -0.01，每恢复一颗心给予 +0.01；但我们没有进一步研究这一设计是否有帮助。

补充材料

Minecraft 视频预测

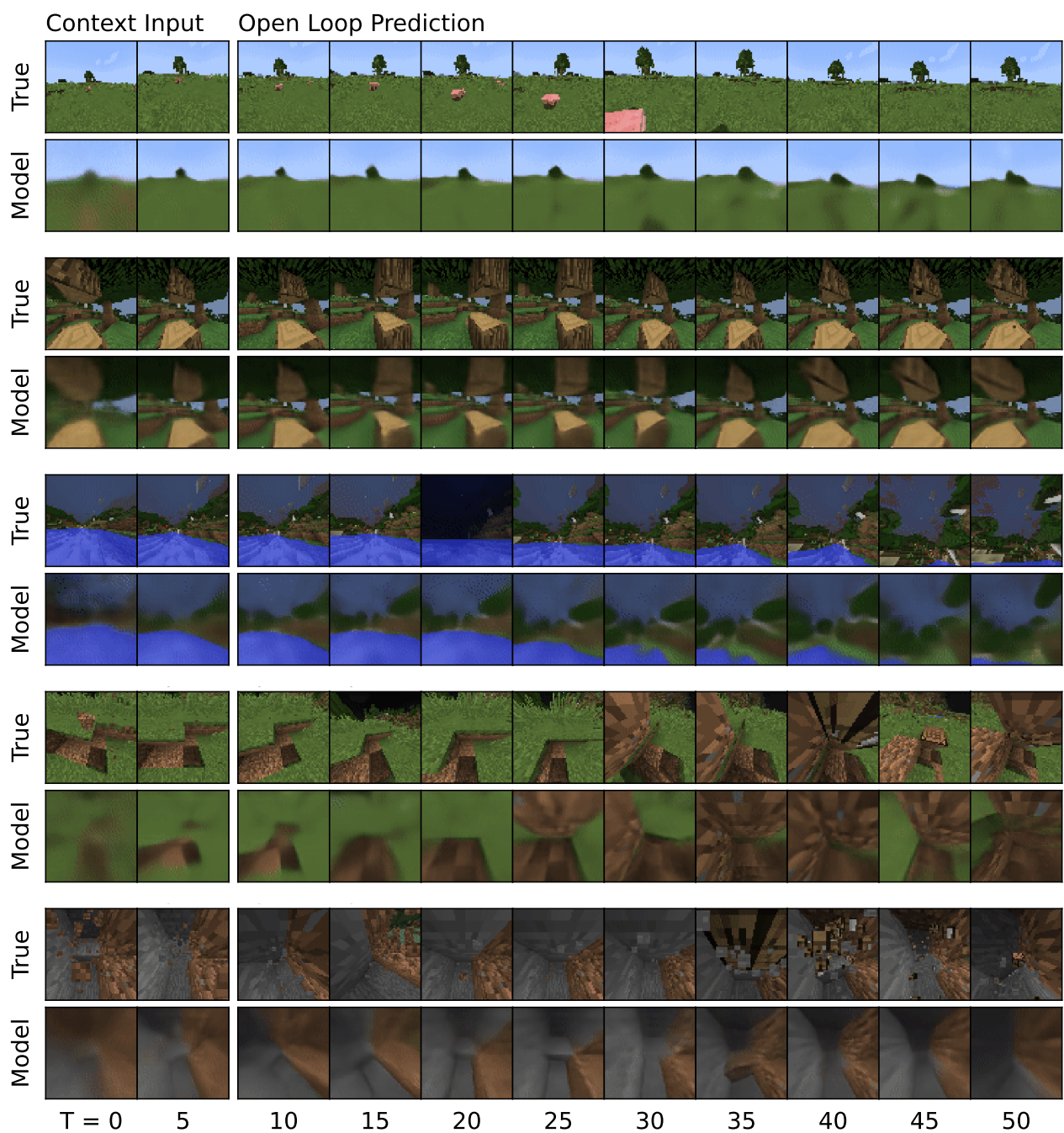


图 7: 在 Minecraft 上进行多步预测。世界模型作为上下文输入接收前 5 帧, 鉴于动作序列且无法获取中间图像, 预测未来将有 45 步。

其他结果

方法	返回时
Dreamer	9.1
意大利	7.1
Rainbow	6.3
PPO	5.1

表 5: Minecraft Diamond scores at 100M environment steps.

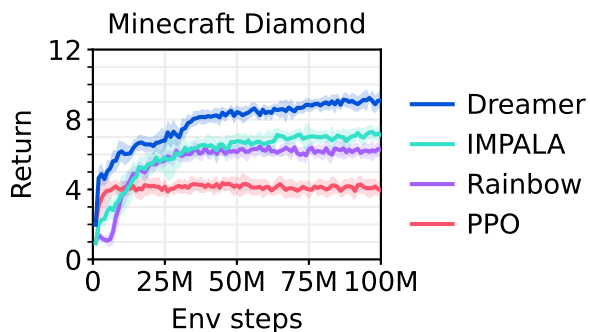


图 8: Minecraft learning curves.

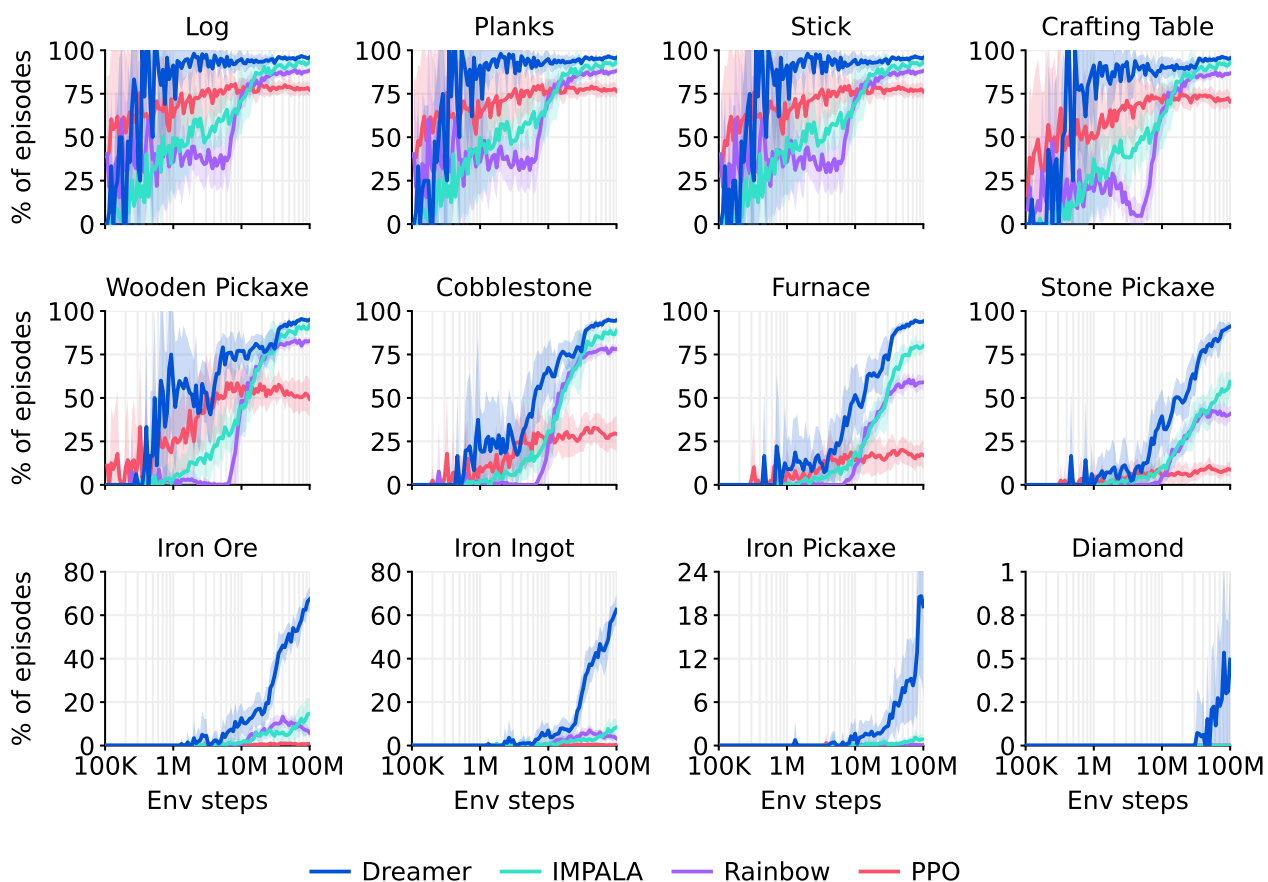


图 9: 物品成功率，按 episode 百分比统计。Dreamer 获得各类物品的比例显著高于基线，并在 100M 步预算结束前持续改进。在该预算下，Dreamer 有 0.4% 的 episode 获得钻石，仍为后续研究留下挑战。该指标不同于图 5：后者显示，在整个训练过程中，100% 的 Dreamer 智能体都至少获得过一次钻石，不受 episode 边界影响；相比之下，基线智能体为 0%。

Atari 学习曲线

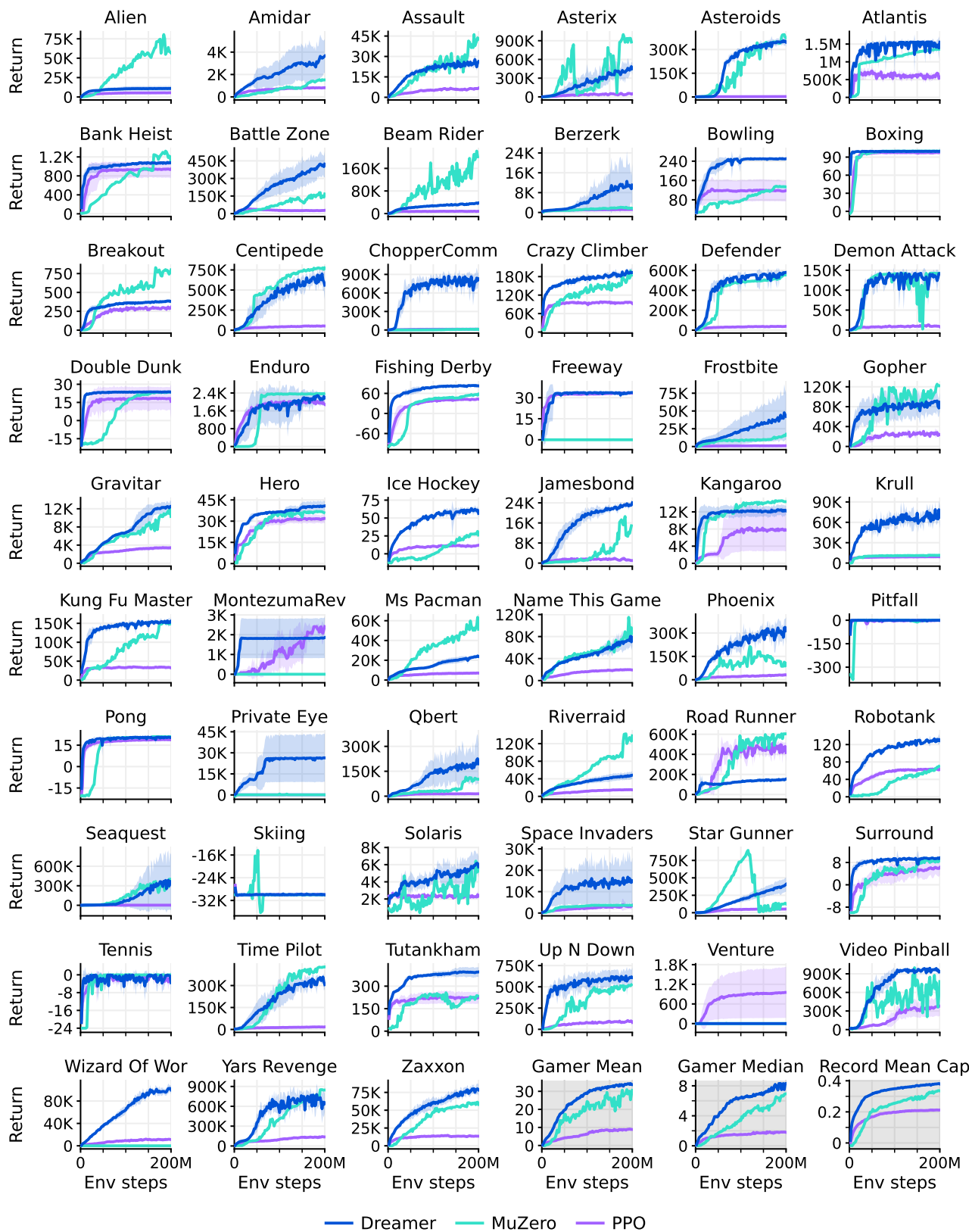


图 10: Atari 学习曲线。

Atari 分数

任务	随机	人类玩家	纪录	PPO	MuZero	Dreamer
环境步数	—	—	—	200M	200M	200M
Alien	228	7128	251916	5476	56835	10977
Amidar	6	1720	104159	817	1517	3612
Assault	222	742	8647	6673	42742	26010
Asterix	210	8503	1000000	47190	879375	441763
Asteroids	719	47389	10506650	2479	374146	348684
Atlantis	12850	29028	10604840	539721	1353617	1553222
Bank Heist	14	753	82058	946	1077	1083
BattleZone	2360	37188	801000	27816	167412	419653
Beam Rider	364	16926	999999	7973	201154	37073
Berzerk	124	2630	1057940	1186	1698	10557
Bowling	23	161	300	118	133	250
Boxing	0	12	100	98	100	100
Breakout	2	30	864	299	799	384
Centipede	2091	12017	1301709	51833	774421	554553
Chopper Command	811	7388	999999	12667	8945	802698
Crazy Climber	10780	35829	219900	93176	184394	193204
Defender	2874	18689	6010500	38270	554492	579875
Demon Attack	152	1971	1556345	8229	142509	142109
Double Dunk	-19	-16	22	16	23	24
Enduro	0	860	9500	1887	2369	2166
Fishing Derby	-92	-39	71	43	58	82
Freeway	0	30	38	33	0	34
Frostbite	65	4335	454830	1123	17087	41888
Gopher	258	2412	355040	24792	122025	87600
Gravitar	173	3351	162850	3436	10301	12570
Hero	1027	30826	1000000	31967	36063	40677
Ice Hockey	-11	1	36	12	26	57
James Bond	29	303	45550	1019	14872	24010
Kangaroo	52	3035	1424600	7769	14380	12229
Krull	1598	2666	104100	9193	11476	69858
Kung-Fu Master	258	22736	1000000	32335	148936	154893
Montezuma Revenge	0	4753	1219200	2368	0	1852
Ms Pacman	307	6952	290090	7041	51310	24079
Name This Game	2292	8049	25220	19441	85331	77809
Phoenix	761	7243	4014440	31412	105593	316606
Pitfall	-229	6464	114000	-2	0	0
Pong	-21	15	21	19	21	20
Private Eye	25	69571	101800	73	100	26432
Qbert	164	13455	2400000	14554	102129	201084
Riverraid	1338	17118	1000000	14860	137983	48080
Road Runner	12	7845	2038100	423995	604083	150402
Robotank	2	12	76	63	70	132
Seaquest	68	42055	999999	1927	399764	356584
Skiing	-17098	-4337	-3272	-29926	-30000	-29965
Solaris	1236	12327	111420	2368	5860	5851
Space Invaders	148	1669	621535	3489	3639	15005
Star Gunner	664	10250	77400	53439	127417	408961
Surround	-10	6	6	6	9	9
Tennis	-24	-8	21	-1	0	-3
Time Pilot	3568	5229	65300	17250	427209	314947
Tutankham	11	168	5384	225	235	395
Up N Down	533	11693	82840	83743	522962	614065
Venture	0	1188	38900	953	0	0
Video Pinball	16257	17668	89218328	382306	775304	940631
Wizard Of Wor	564	4756	395300	10910	0	99136
Yars Revenge	3093	54577	15000105	137164	846061	675774
Zaxxon	32	9173	83700	13599	58115	78443
人类归一化中位数 (%)	0	100	3716	180	693	830
人类归一化平均值 (%)	0	100	123001	892	3054	3381
纪录归一化平均值 (%)	0	13	100	21	66	74
纪录归一化平均值 (截断) (%)	0	13	100	21	34	38

表 6: Atari 分数。

ProcGen 学习曲线

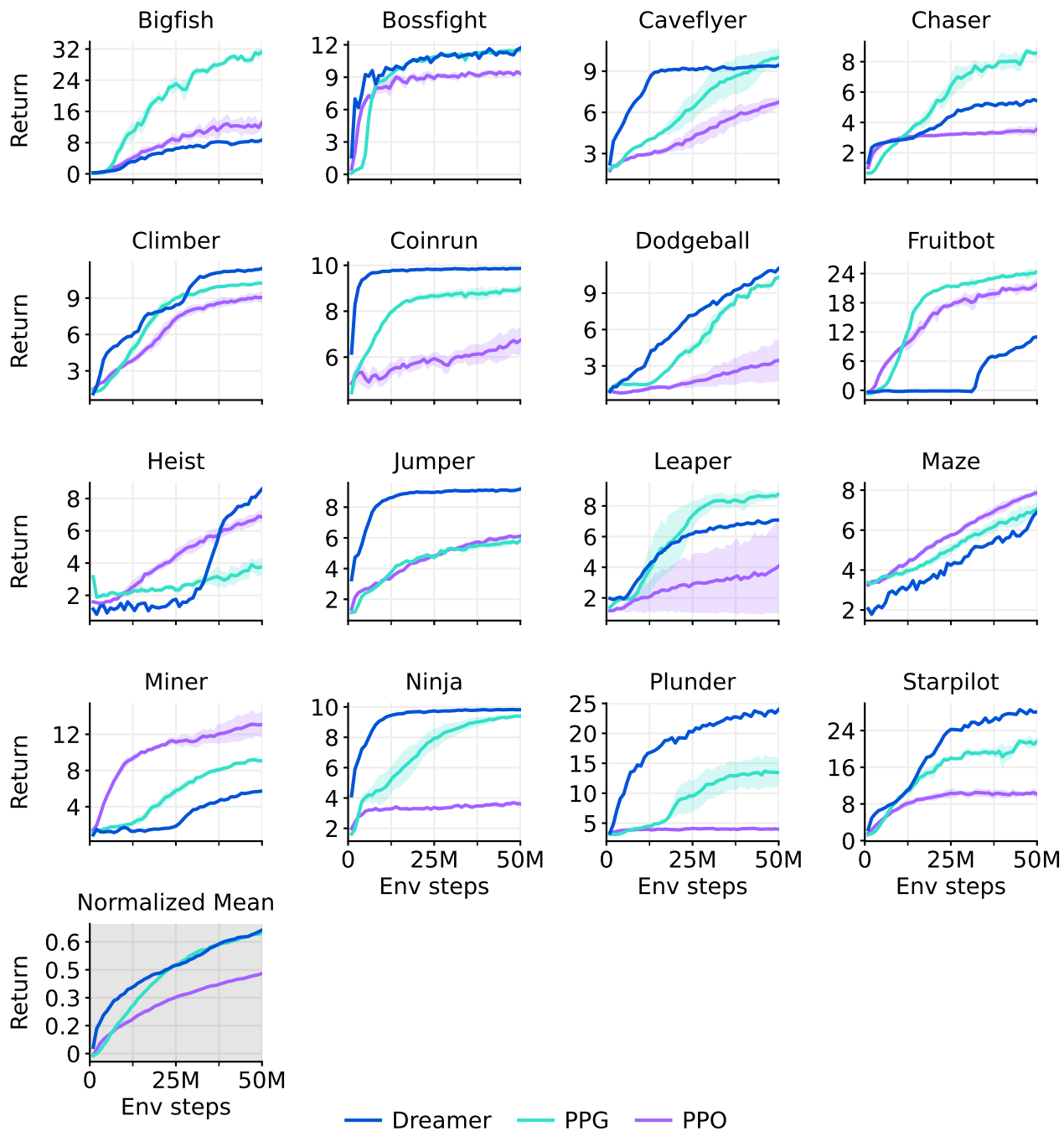


图 11: ProcGen 学习曲线.

ProcGen 分数

任务	Original PPO	PPO	PPG	Dreamer
环境步数	50M	50M	50M	50M
Bigfish	10.92	12.72	31.26	8.62
Bossfight	10.47	9.36	11.46	11.61
Caveflyer	6.03	6.71	10.02	9.42
Chaser	4.48	3.54	8.57	5.49
Climber	7.59	9.04	10.24	11.43
Coinrun	7.93	6.71	8.98	9.86
Dodgeball	4.80	3.44	10.31	10.93
Fruitbot	20.28	21.69	24.32	11.04
Heist	2.25	6.87	3.77	8.51
Jumper	5.09	6.13	5.84	9.17
Leaper	5.90	4.07	8.76	7.05
Maze	4.97	7.86	7.06	6.85
Miner	7.56	12.97	9.08	5.71
Ninja	6.16	3.62	9.38	9.82
Plunder	11.16	3.99	13.44	23.81
Starpilot	17.00	10.13	21.57	28.00
归一化均值	41.16	42.80	64.89	66.01

表 7: ProcGen 分数。本文使用的固定超参数 PPO 实现与原始 PPO 持平或更好；原始 PPO 作者称其经过高度调优，接近最优超参数³⁷。

DMLab 学习曲线

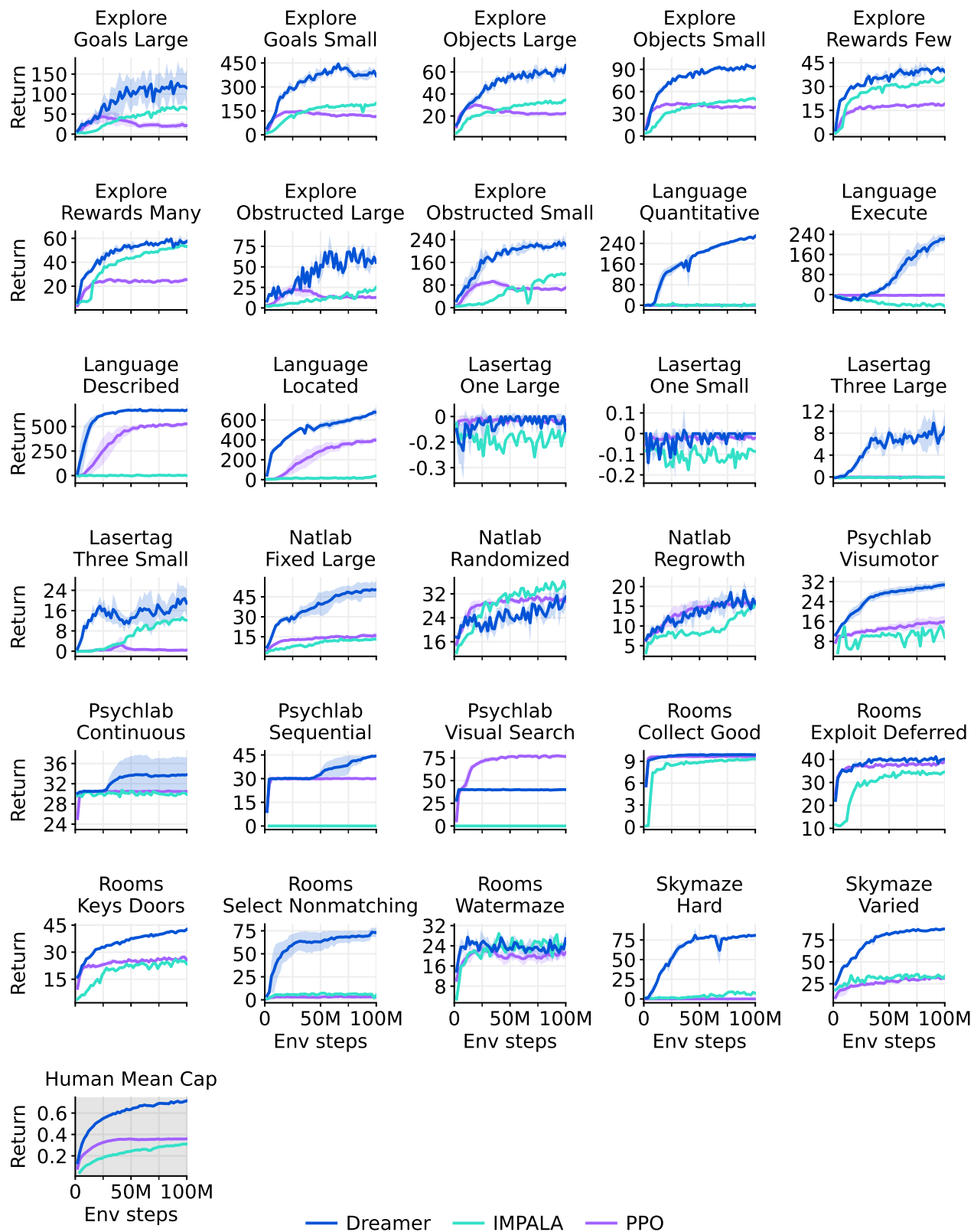


图 12: DMLab 学习曲线.

DMLab 分数

任务	R2D2+	IMPALA	IMPALA	IMPALA	PPO	Dreamer
环境步数	10B	10B	1B	100M	100M	100M
Explore Goal Locations Large	174.7	316.0	137.8	64.2	21.2	116.7
Explore Goal Locations Small	460.7	482.0	302.8	196.1	115.1	372.8
Explore Object Locations Large	60.6	91.0	55.1	34.3	22.5	63.9
Explore Object Locations Small	83.7	100.4	75.9	50.6	38.9	93.5
Explore Object Rewards Few	80.7	92.6	46.9	34.6	19.0	40.0
Explore Object Rewards Many	75.8	89.4	68.5	53.3	25.5	58.1
Explore Obstructed Goals Large	95.5	102.0	57.9	23.7	12.9	52.8
Explore Obstructed Goals Small	311.9	372.0	214.9	118.0	70.4	224.6
Language Answer Quantitative Question	344.4	362.0	304.7	1.0	0.3	266.0
Language Execute Random Task	497.4	465.4	140.8	-44.4	-2.5	223.7
Language Select Described Object	617.6	664.0	618.4	0.2	526.9	665.5
Language Select Located Object	772.8	731.4	413.0	38.2	397.5	679.5
Lasertag One Opponent Large	0.0	0.0	0.0	-0.1	0.0	-0.1
Lasertag One Opponent Small	31.8	0.0	0.0	-0.1	0.0	0.0
Lasertag Three Opponents Large	28.6	32.2	10.4	-0.1	0.0	9.0
Lasertag Three Opponents Small	49.0	57.2	37.1	12.1	0.4	18.8
Natlab Fixed Large Map	60.6	63.4	53.8	13.0	15.8	50.5
Natlab Varying Map Randomized	42.4	47.0	40.5	35.3	29.6	31.2
Natlab Varying Map Regrowth	24.6	34.0	25.5	15.3	16.3	16.7
Psychlab Arbitrary Visuomotor Mapping	33.1	38.4	16.5	11.9	16.0	30.7
Psychlab Continuous Recognition	30.0	28.6	30.0	30.1	30.5	33.8
Psychlab Sequential Comparison	30.0	29.6	0.0	0.0	30.0	44.3
Psychlab Visual Search	79.9	80.0	0.0	0.0	76.6	40.1
Rooms Collect Good Objects Test	9.9	10.0	9.9	9.3	9.7	9.9
Rooms Exploit Deferred Effects Test	38.1	62.2	37.6	34.5	39.0	40.4
Rooms Keys Doors Puzzle	46.2	54.6	36.9	24.2	26.0	42.4
Rooms Select Nonmatching Object	63.6	39.0	63.2	4.0	2.7	73.1
Rooms Watermaze	49.0	47.0	50.1	23.6	21.2	26.1
Skymaze Irreversible Path Hard	76.0	80.0	46.4	7.7	0.0	80.9
Skymaze Irreversible Path Varied	76.0	100.0	69.8	32.7	31.3	87.7
人类均值截断分数 (%)	85.4	85.1	66.3	31.0	35.9	71.4

表 8: DMLab 在 100M 环境步和更大预算下的分数。IMPALA 智能体对应 Kapturowski et al.³⁵ 报告的 “IMPALA (deep)”；该工作公开了学习曲线。

Atari100k 学习曲线

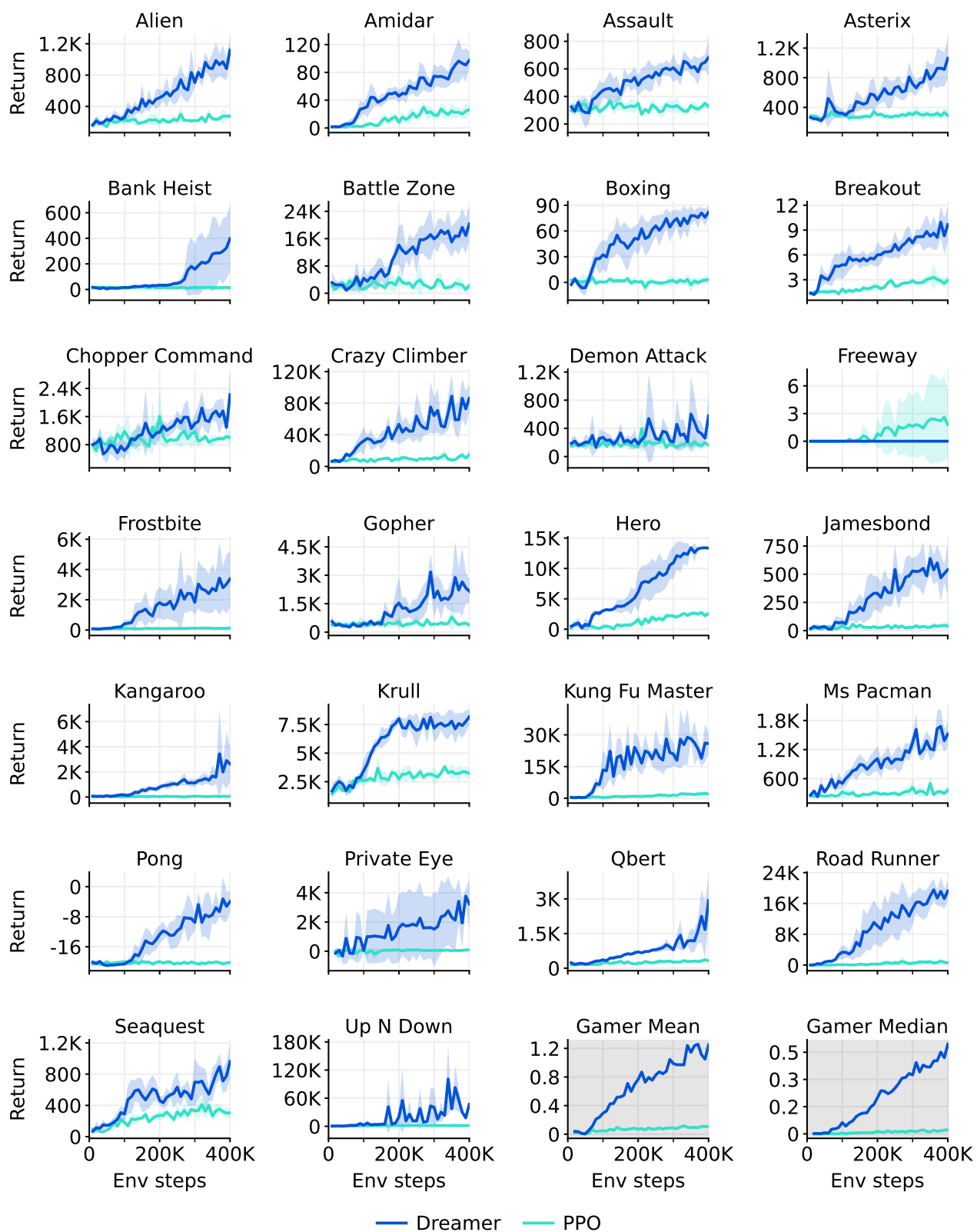


图 13: Atari100k 学习曲线.

Atari100 分的成绩

任务	随机	人类	PPO	SimPLe	SPR	TWM	IRIS	Dreamer
环境步数	—	—	400K	400K	400K	400K	400K	400K
Alien	228	7128	276	617	842	675	420	1118
Amidar	6	1720	26	74	180	122	143	97
Assault	222	742	327	527	566	683	1524	683
Asterix	210	8503	292	1128	962	1117	854	1062
Bank Heist	14	753	14	34	345	467	53	398
BattleZone	2360	37188	2233	4031	14834	5068	13074	20300
Boxing	0	12	3	8	36	78	70	82
Breakout	2	30	3	16	20	20	84	10
Chopper	811	7388	1005	979	946	1697	1565	2222
Command								
Crazy Climber	10780	35829	14675	62584	36700	71820	59324	86225
Demon Attack	152	1971	160	208	518	350	2034	577
Freeway	0	30	2	17	19	24	31	0
Frostbite	65	4335	127	237	1171	1476	259	3377
Gopher	258	2412	368	597	661	1675	2236	2160
Hero	1027	30826	2596	2657	5859	7254	7037	13354
James Bond	29	303	41	100	366	362	463	540
Kangaroo	52	3035	55	51	3617	1240	838	2643
Krull	1598	2666	3222	2205	3682	6349	6616	8171
Kung-Fu Master	258	22736	2090	14862	14783	24555	21760	25900
Ms Pacman	307	6952	366	1480	1318	1588	999	1521
Pong	-21	15	-20	13	-5	19	15	-4
Private Eye	25	69571	100	35	86	87	100	3238
Qbert	164	13455	317	1289	866	3331	746	2921
Road Runner	12	7845	602	5641	12213	9109	9615	19230
Seaquest	68	42055	305	683	558	774	661	962
Up N Down	533	11693	1502	3350	10859	15982	3546	46910
人类归一化平均值 (%)	0	100	11	33	62	96	105	125
人类归一化中位数 (%)	0	100	2	13	40	51	29	49

表 9: Atari100k 在 400K 环境步数下的得分，对应 100K 智能体步数。

设置	SimPLe	EffMuZero	SPR	IRIS	TWM	Dreamer
人类归一化平均值 (%)	33	190	62	105	96	125
人类归一化中位数 (%)	13	109	40	29	51	49
GPU 日数	5.0	0.6	0.1	3.5	0.4	0.1
在线规划	—	X	—	—	—	—
数据增强	—	—	X	—	—	—
非均匀回放	—	X	X	—	X	—
单独超参数	—	—	—	X	—	—
增加分辨率	—	X	X	—	—	—
使用生命信息	—	X	X	X	X	—
提前重置关卡	—	X	—	X	—	—
单独的 eval 回合	X	X	X	X	X	—

表 10: Atari100k 基准的评估协议。计算资源换算为 A100 GPU 天数。EfficientMuZero⁴⁴ 取得最高分，但相对标准设置¹⁷ 修改了环境配置。IRIS 在 Freeway 上为探索强度使用了单独超参数。

主动控制学习曲线

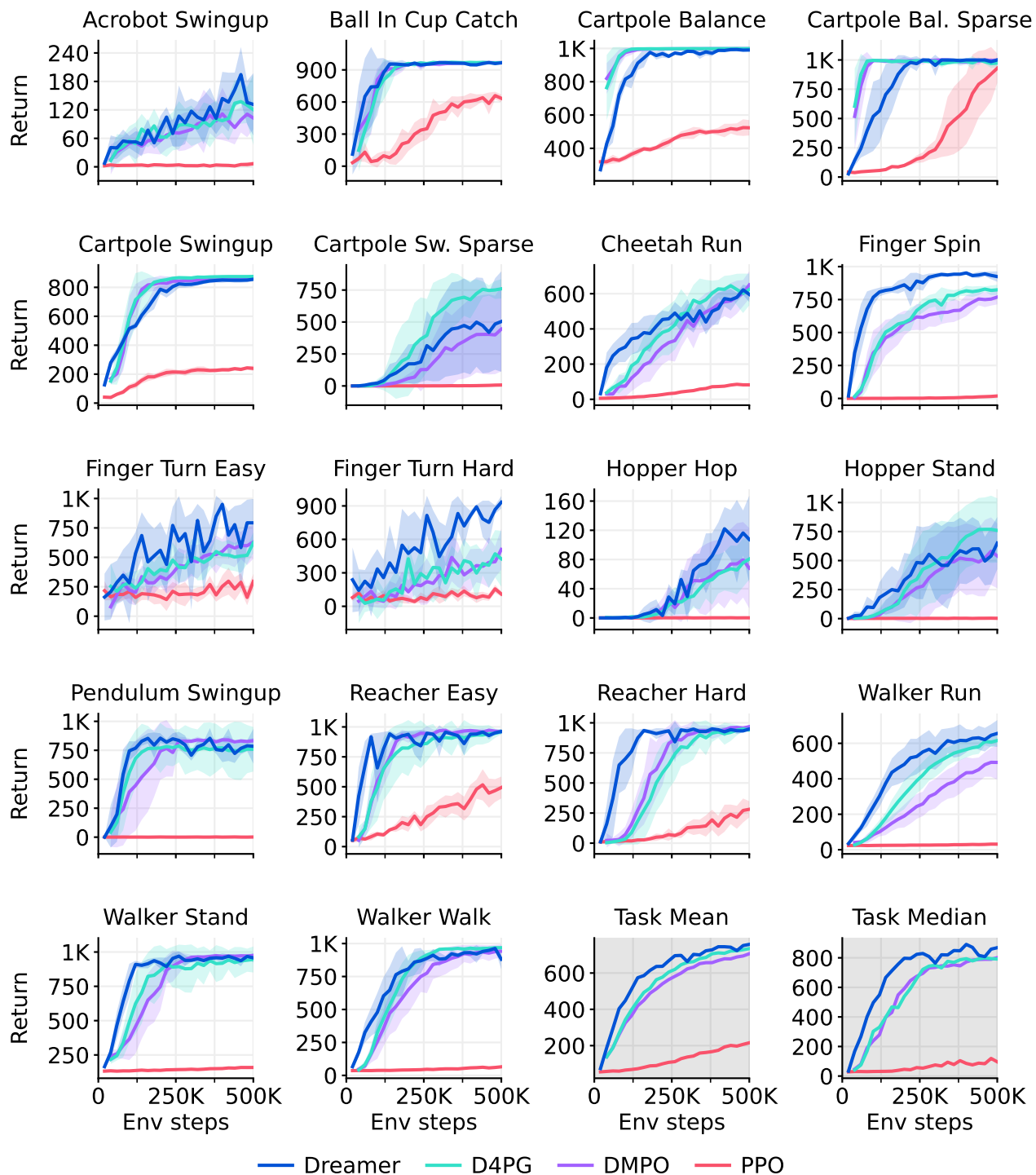


图 14: DeepMind Control Suite 自主输入下的学习曲线。

主动控制分数

任务	PPO	DDPG	DMPO	D4PG	Dreamer
环境步数	500K	500K	500K	500K	500K
Acrobot Swingup	6	100	103	124	134
Ball In Cup Catch	632	917	968	968	962
Cartpole Balance	523	997	999	999	990
Cartpole Balance Sparse	930	992	999	974	990
Cartpole Swingup	240	864	860	875	852
Cartpole Swingup Sparse	7	703	438	752	491
Cheetah Run	82	596	650	624	614
Finger Spin	18	775	769	823	931
Finger Turn Easy	281	499	620	612	793
Finger Turn Hard	106	313	495	421	889
Hopper Hop	0	36	68	80	113
Hopper Stand	3	484	549	762	576
Pendulum Swingup	1	767	834	759	788
Reacher Easy	494	934	961	960	954
Reacher Hard	288	949	968	937	938
Walker Run	31	561	493	616	649
Walker Stand	159	965	975	947	964
Walker Walk	64	952	942	969	936
任务均值	94	771	801	792	871
任务中位数	215	689	705	733	754

表 11: 本体感知输入设置下的 DeepMind Control Suite 分数。

视觉控制学习曲线

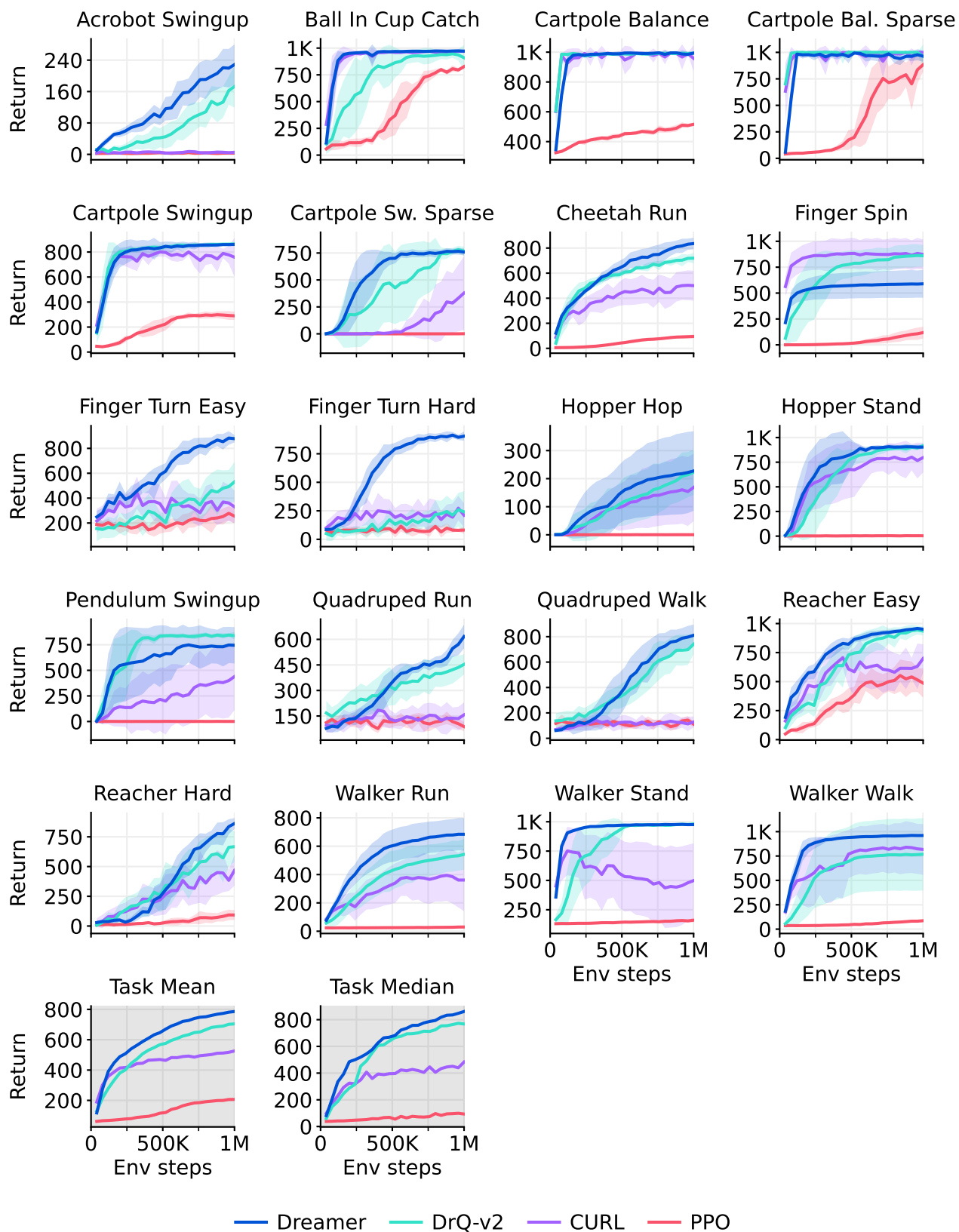


图 15: DeepMind Control Suite 视觉输入下的学习曲线。

视觉控制分数

任务	PPO	SAC	CURL	DrQ-v2	Dreamer
环境步数	1M	1M	1M	1M	1M
Acrobot Swingup	3	4	4	166	229
Ball In Cup Catch	829	176	970	928	972
Cartpole Balance	516	937	980	992	993
Cartpole Balance Sparse	881	956	999	987	964
Cartpole Swingup	290	706	771	863	861
Cartpole Swingup Sparse	1	149	373	773	759
Cheetah Run	95	20	502	716	836
Finger Spin	118	291	880	862	589
Finger Turn Easy	253	200	340	525	878
Finger Turn Hard	79	94	231	247	904
Hopper Hop	0	0	164	221	227
Hopper Stand	4	5	777	903	903
Pendulum Swingup	1	592	413	843	744
Quadruped Run	88	54	149	450	617
Quadruped Walk	112	49	121	726	811
Reacher Easy	487	67	689	944	951
Reacher Hard	94	7	472	670	862
Walker Run	30	27	360	539	684
Walker Stand	161	143	486	978	976
Walker Walk	87	40	822	768	961
任务均值	94	81	479	770	861
任务中位数	206	226	525	705	786

表 12: 视觉输入设置下的 DeepMind Control Suite 分数。

BSuite 性能谱

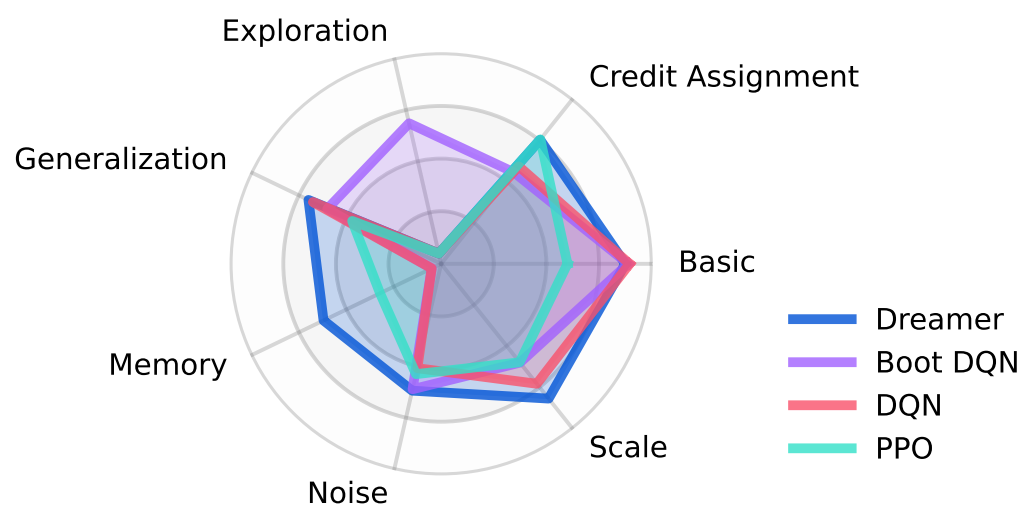


图 16: 按类别可视化的 BSuite 分数⁴⁸。Dreamer 大于类别表和内存中的以往方法。

BSuite 分数

任务	Random	PPO	AC-RNN	DQN	Boot DQN	Dreamer
Bandit	0.00	0.38	1.00	0.93	0.98	0.96
Bandit Noise	0.00	0.61	0.63	0.71	0.80	0.75
Bandit Scale	0.00	0.39	0.60	0.74	0.83	0.78
Cartpole	0.04	0.84	0.40	0.85	0.69	0.93
Cartpole Noise	0.04	0.77	0.20	0.82	0.69	0.93
Cartpole Scale	0.04	0.83	0.12	0.72	0.65	0.92
Cartpole Swingup	0.00	0.00	0.00	0.00	0.15	0.03
Catch	0.00	0.91	0.87	0.92	0.99	0.96
Catch Noise	0.00	0.54	0.27	0.58	0.68	0.53
Catch Scale	0.00	0.90	0.17	0.85	0.65	0.94
Deep Sea	0.00	0.00	0.00	0.00	1.00	0.00
Deep Sea Stochastic	0.00	0.00	0.00	0.00	0.90	0.00
Discounting Chain	0.20	0.24	0.39	0.25	0.22	0.40
Memory Len	0.00	0.17	0.70	0.04	0.04	0.65
Memory Size	0.00	0.47	0.29	0.00	0.00	0.59
Mnist	0.05	0.77	0.56	0.85	0.85	0.61
Mnist Noise	0.05	0.41	0.22	0.38	0.34	0.34
Mnist Scale	0.05	0.76	0.09	0.49	0.31	0.55
Mountain Car	0.10	0.10	0.10	0.93	0.93	0.92
Mountain Car Noise	0.10	0.10	0.10	0.89	0.82	0.87
Mountain Car Scale	0.10	0.10	0.10	0.85	0.56	0.90
Umbrella Distract	0.00	1.00	0.09	0.30	0.26	0.74
Umbrella Length	0.00	0.87	0.43	0.39	0.39	0.78
Basic	0.04	0.60	0.58	0.90	0.89	0.88
Credit assignment	0.03	0.76	0.37	0.59	0.56	0.75
Exploration	0.00	0.00	0.00	0.00	0.68	0.01
Generalization	0.06	0.47	0.19	0.68	0.60	0.70
Memory	0.00	0.32	0.49	0.02	0.02	0.62
Noise	0.02	0.54	0.24	0.51	0.61	0.62
Scale	0.04	0.60	0.22	0.73	0.60	0.82
任务均值 (%)	3	49	32	54	60	66
类别均值 (%)	3	47	30	49	57	63

表 13: BSuite 各任务分数。表中给出每个任务在环境配置上的平均值，以及按类别和全任务聚合后的表现。

强壮的爆发

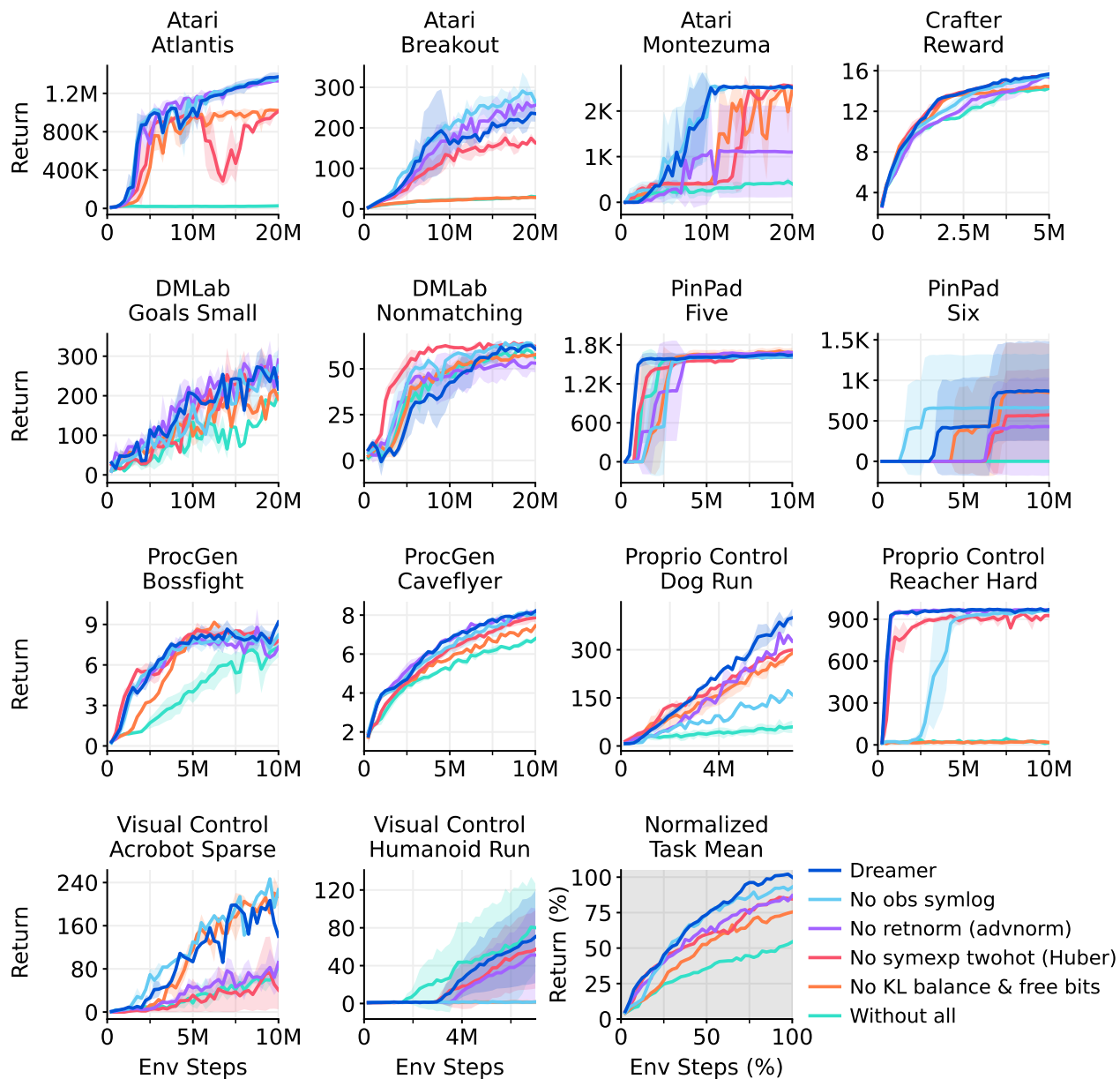


图 17: 用于强性衰减实验的单个学习曲线。所有强性技术都有助于总体性能 Dreamer，虽然每种个别技术只能改进对某一子任务的表现。

学习信号

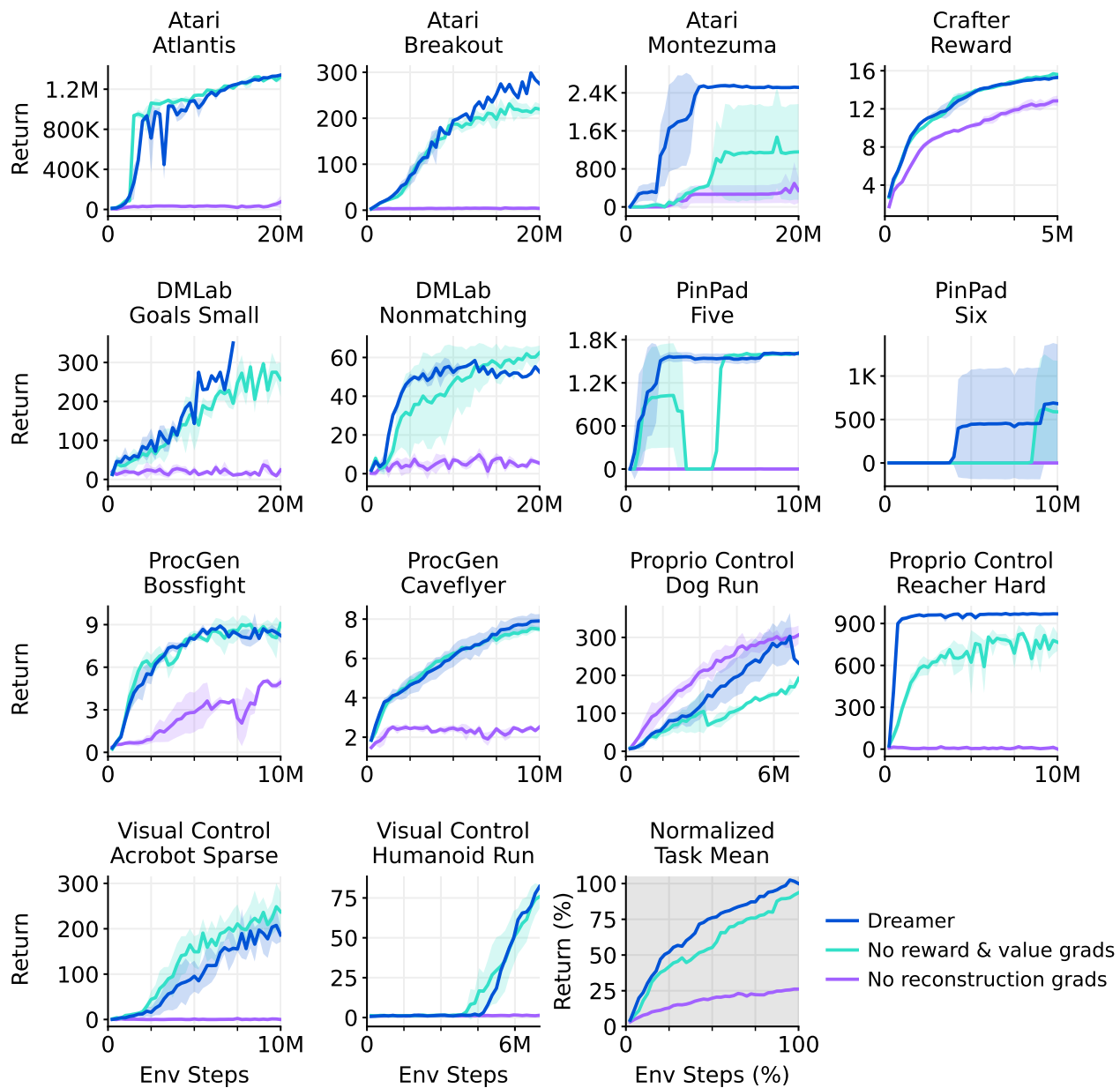


图 18: 个别学习曲线用于学习信号衰减实验. Dreamer 主要依靠其世界模型中监督不足的重建目标, 以及额外的奖励和值梯度, 进一步提高了一组任务的性能。